

HYDRA 🐉: A NAVIGATION WORLD ACTION MODEL WITH DISCRETE LATENT PLANNING AND CONTINUOUS FLOW-MATCHING EXECUTION

Mohammad Nazeri* Alexandyr Card*[¶] Samira Huber[†][¶] Anuj Pokhrel*[¶] Yujun Wang[‡][¶]
 Ruben Hammele[†][¶] Daeun Song[§] Sören Pirk[†] Xuesu Xiao*

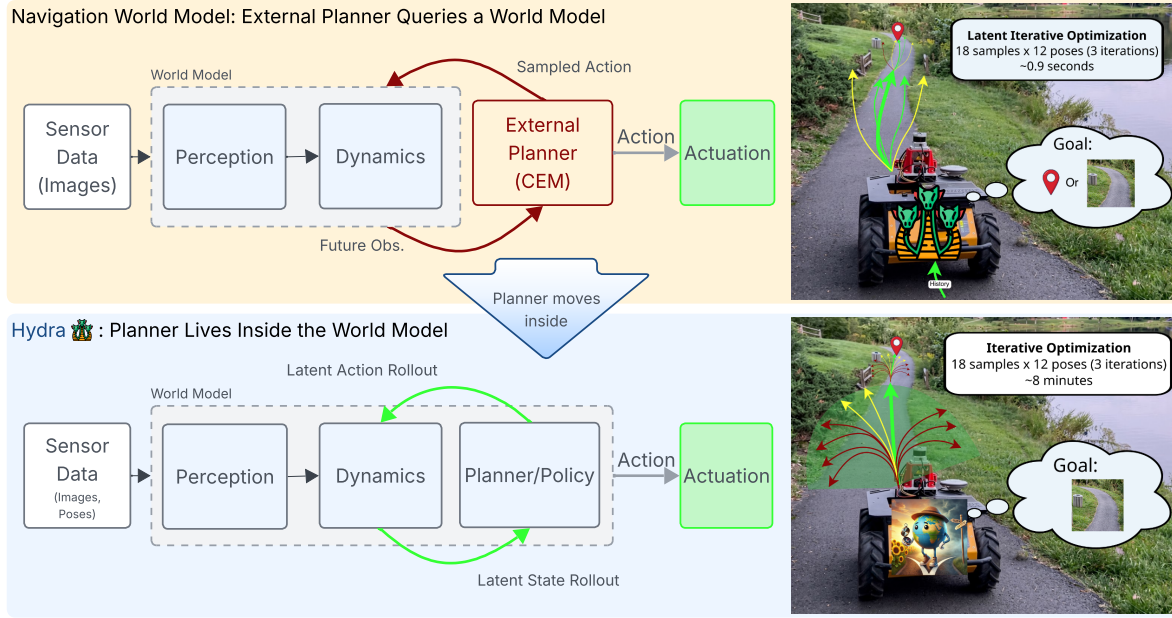


Figure 1: **Overcoming the continuous sampling bottleneck in world models.** **Top:** Navigation World Model (NWM) blindly evaluates unconstrained continuous trajectories, wasting compute on bad trajectories (red) and failing real-time constraints. **Bottom:** Hydra introduces *Discrete* Latent Planning and *continuous* action execution by Flow Matching. By restricting its search space to a learned manifold of valid topological priors and evaluating them via a Kinematic-Perceptual Cost, Hydra rapidly plans paths and achieves $> 500\times$ computational efficiency onboard a physical robot.

ABSTRACT

World models let robots imagine possible futures, but exploiting this capability for real-time control is bottlenecked by a *representation misalignment*: the generative model and the planner operate on decoupled manifolds, so the planner has no shared structure to search over and must instead decode every candidate back into high-dimensional pixel space to evaluate it. This decoding step is a major obstacle to real-time control on physical hardware. In this paper, we present Hydra, a discrete World Action Model that closes this gap by moving the planner, both the sampler and the evaluator, inside the model. Hydra establishes a unified latent manifold over visual states, physical poses, and control

*George Mason University, {mnazerir, acard, apokhrel, xiao}@gmu.edu

[†]Kiel University, {samira.huber, ruben.hammele, sp}@informatik.uni-kiel.de

[‡]Ludwig Maximilian University Munich, yujun_wang_cn@hotmail.com

[§]Ewha Womans University, songd@ewha.ac.kr

[¶]Equal contribution as second authors.

actions, then compresses this manifold through modality-specific Vector-Quantized bottlenecks into discrete vocabularies of kinodynamic intents and visual states. Because candidates are now drawn directly from this shared manifold, sampling is informed by the model’s own understanding of the observation rather than proposed blind, and evaluation happens natively within the discrete space: candidates are ranked by a Kinematic-Perceptual Cost, without ever decoding to pixels. We term this *Discrete Latent Planning* (DLP). Because planning over discrete intents alone cannot supply the smooth, continuous commands physical actuation requires, Hydra pairs DLP with conditional Flow Matching, which maps each selected intent to a continuous trajectory for execution. Evaluated on two physical robotic platforms, Hydra outperforms state-of-the-art world models in goal-directed planning, while matching or exceeding the closed-loop execution capabilities of leading reactive foundation policies⁵.

1 Introduction

Visual robot navigation maps high-dimensional ego-centric observations directly to low-level motor commands. Foundation navigation policies [57, 58, 61] perform this mapping effectively in structured environments, but remain fundamentally reactive: without an explicit internal model of world dynamics, they cannot simulate the consequences of a candidate action before taking it. This leaves them myopically focused on immediate visual features, making them susceptible to local minima (*e.g.*, dead ends) and incapable of planning.

To endow robots with foresight, focus has shifted toward World Models (WMs) [20], such as the Navigation World Model (NWM) [6], which predict future states conditioned on action sequences proposed by an external planner such as CEM [54] or MPPI [74]. Because the generative model and the planner operate on decoupled manifolds – a *representation misalignment* (Fig. 1) – the planner has no shared structure with the model to search over, and so must resort to uninformed sampling over an unconstrained space. This space grows intractably large with horizon length, forcing the planner to either sample too sparsely to find good candidates or spend prohibitive compute exploring implausible ones. Additionally, because sampled candidates are not directly comparable to the goal in this space, the planner must additionally decode every candidate’s latent state back into raw pixels, scoring it against the goal with perceptual metrics such as LPIPS [82] that are sensitive to visual distractors rather than true spatial distance.

To address this misalignment and enable planning for mobile robots, we present Hydra, a World Action Model (WAM) that moves the planner itself, both the sampler and the evaluator, inside the model. Hydra establishes a *unified latent manifold* wherein world and robot dynamics, alongside action policies, are jointly learned, so that action-induced changes in the world are tracked by corresponding, consistent shifts in the latent representation. Sampling now draws directly from this jointly learned manifold, so candidate actions are informed by the model’s own understanding of the observation; the manifold is bounded, keeping the search manageable across longer horizons; and evaluation happens within this shared latent space, so candidates can be scored without decoding back to pixels.

To bypass the bottleneck of continuous sampling, Hydra forces its future predictions through modality-specific Vector Quantized (VQ) [70] codebooks, compressing the unbounded, continuous action space into a discrete manifold of physically feasible *intents* – macro-level kinodynamic primitives (*e.g.*, a sweeping left turn) drawn from a small, learned vocabulary. This substitution changes what planning search actually is: rather than optimizing over an unbounded continuous space in which some points are implausible, Hydra searches over a finite set of behaviors already known to be physically plausible, with the code retrieved at each step being the one whose embedding lies closest to Hydra’s embedding from the observation history, rather than an arbitrary vocabulary entry. We term this search *Discrete Latent Planning* (DLP). As Fig. 1 illustrates, this changes not only how many candidates are considered but what kind: NWM’s continuous sampler (red) spreads trajectories indiscriminately, including clearly bad ones, but must evaluate every one to find this out; Hydra’s candidates (green) are physically plausible by construction.

This discreteness also makes candidate evaluation simple. Because each predicted future intent – kinodynamic or visual – is a draw from a categorical distribution over its respective codebook, its Shannon entropy is a cheap measure of predictive uncertainty, and its quantization error is a measure of how far a candidate is from anything the model was trained on – a signal a continuous, decoupled action space cannot provide without additional machinery such as ensembles or an explicit density model. DLP exploits this directly: candidates are ranked by a Kinematic-Perceptual Cost $\mathcal{J}_{KPC}$ (Eq. (4)) that detects impending collisions as out-of-distribution (OOD) quantization error in the *visual* codebook, evaluated entirely within the discrete manifold and without ever decoding to pixels or querying a separate obstacle-detection model (Sec. 3.3). The winning candidate is then handed to continuous Flow Matching ODE solvers, which integrate DLP-selected intent – *e.g.*, a discrete 40° turn – into the continuous pose or action required for smooth physical execution.

⁵Model weights and deployment code are available at the project website: <https://robotixx.github.io/hydra>.

In summary, our main contributions are: 1) **Hydra**: A World Action Model that, unlike *in-context alignment methods* that burden the Transformer backbone with aligning disparate modality tokens in-context, leverages an *early alignment* by fusing the visual and kinodynamic inputs. The unified representation models the input modalities into a singular latent before sequence modeling, and dedicates the entirety of the autoregressive capacity to temporal dynamics, resulting in a smaller model size. 2) **Discrete Latent Planning**: A novel iterative refinement latent search utilizing a hybrid Kinematic-Perceptual Cost that bypasses the computational waste of continuous unconstrained sampling. 3) **Kinematic-Perceptual Cost**: A novel multi-objective MPC framework that evaluates trajectory safety entirely within the discrete latent manifold. 4) **Real-World Deployment**: We demonstrate that **Hydra** functions as both a standalone global planner and a reactive local path follower, achieving better planning performance than baselines (NWM [6]) while matching the execution precision of dedicated behavior cloning policies.

2 Related Work

To show the contributions of **Hydra**, we review prior literature across four dimensions: *Reactive Navigation Policies*, which lack the generative foresight required for planning; *Generative World Models*, which predict but often lack explicit Euclidean grounding; *Planning with a Model*, where standard continuous sampling imposes computational bottlenecks that **Hydra**’s discrete latent planning resolves; and *in-context alignment versus early alignment*, where early fusion saves model capacity for dynamics learning only.

Reactive Navigation Policies: Driven by large-scale heterogeneous datasets, recent efforts have produced highly capable end-to-end navigation policies [57, 58, 61, 37]. Furthermore, advancements in continuous generative control, most notably Diffusion Policy [16, 45, 85, 49], have demonstrated remarkable success in modeling multimodal action distributions. However, these architectures are fundamentally reactive, compressing navigation into a direct observation-to-action mapping. Because they are open-loop with respect to the future environment, they cannot predict the consequences of their actions, making them myopic and incapable of evaluating or pruning trajectories before execution.

Generative World Models: To endow robots with foresight, WMs [20, 76, 26, 80, 47, 50, 89, 41, 30, 46] simulate the consequences of actions. Early predict-then-act frameworks [17, 73, 9] synthesized video trajectories before a separate policy derived actions, introducing high latency and physical inconsistencies. Recent approaches have shifted toward unified predictive structures via diffusion [60, 53, 24] or Flow Matching (FM) [36, 33, 77]. While visual-action models [25, 55, 6, 12] and large-scale unified backbones [32, 88, 2, 48, 42, 75, 65, 66] provide tighter coupling, they typically do so by scaling to billions of parameters and targeting general-purpose manipulation. DreamZero [78], for instance, uses a 14B-parameter WAM as a zero-shot policy on physical manipulators (AgiBot G1, Franka), but critically, it performs no search: it maps observations directly to actions in one pass, since rollout-based planning at this scale would require prohibitively many forward passes per control step. **Hydra** targets navigation instead, at 143M parameters, sized for the real-time, onboard search we demonstrate in Sec. 4.

Planning with a Model: Planning with world models is traditionally executed via sampling algorithms such as MPPI or CEM [87, 6, 41]; among these, only NWM, VertiFormer, and DINO-WM perform genuine test-time search with evaluative feedback comparable to ours, and for navigation specifically, NWM and VertiFormer [44] (via MPPI) remain the closest baselines. In unconstrained continuous spaces, these planners suffer from severe computational inefficiency, wasting compute on thousands of implausible trajectories. Furthermore, visually evaluating these trajectories with perceptual metrics such as LPIPS [82] can trap planners into texture-matching illusions. To bypass pixel-space rendering overhead, recent frameworks pivot to latent-space planning [18, 84, 62, 87, 59, 4, 63]. While JEPA-style [3, 7, 89, 4, 5] latent models work in latent space, they still evaluate continuous action manifolds, inheriting the sample inefficiency. **Hydra** fundamentally diverges from this paradigm. Discretizing the latent space is not itself new – DreamerV3 [21] and IRIS [43] do so via discrete latents – but neither couples its discretization to a cost function ranking candidates in latent space, nor targets physical, real-time robot planning. By enforcing a VQ intent bottleneck over kinodynamic and visual states jointly, we enable DLP to actively prune unsafe branches without continuous sampling waste, on a physical robot rather than in simulation.

Closest to our planning formulation is WorldPlanner [29], which combines a small diffusion-based world model trained on unstructured play data with a Monte Carlo Tree Search planner and a zeroth-order MPC controller for real-world manipulation. Notably, WorldPlanner discretizes the MCTS search space using a stochastic diffusion model capturing the play distribution. However, this discretization operates on the tree structure and branching of the search itself, not on the underlying representation; actions and states remain continuous and pixel-valued throughout, so evaluating a candidate still requires the decode-then-evaluate paradigm even with a compact, task-specific world model. In contrast, **Hydra** discretizes the representation itself: both kinodynamic intents and visual states are drawn from finite, learned

vocabularies, so search, uncertainty estimation, and cost evaluation are performed entirely within this discrete latent space rather than requiring per-candidate decoding.

Early vs In-context Alignment: Recent robotic foundation models frequently adopt *in-context alignment* (e.g., Gato [52], Cosmos [47, 30, 46]), interleaving disparate modalities (visual patches, proprioception, actions) into a single extended Transformer sequence. While architecturally unified, this in-context alignment forces the self-attention mechanism to simultaneously solve cross-modal alignment and temporal correlations. Consequently, these models require massive scale in data [38] and model size to align multiple modalities; at smaller scales, the structurally disjoint tokens struggle to capture physical violations, leading to modality decoupling and temporal drift [44].

In contrast, **Hydra** aligns with the paradigm of *early alignment*. Unlike FiLM-style affine conditioning (used in RT-1 [11], for language), **Hydra**’s multimodal fusion is a pure additive shift that builds upon VertFormer [44]. This prioritizes a pre-aligned token space. Rather than interleaving modalities, **Hydra** inextricably fuses the observation and action into a single unified token *before* sequence modeling, dedicating its entire capacity to physical dynamics rather than modality alignment, so that any physically impossible sequence is immediately recognized as a violation of the unified manifold.

3 Hydra

This section presents **Hydra** in four parts. We first formalize autonomous navigation as three competing paradigms (see Fig. 2), situating **Hydra**’s unified planning formulation relative to reactive policies and decoupled, external-planner world models (Sec. 3.1). We then describe **Hydra**’s architecture: how visual, pose, and action modalities are fused into a shared representation, and how the resulting intents are fed to a discrete codebook (Sec. 3.2). Building on this representation, we detail how **Hydra** performs Discrete Latent Planning directly within the codebook manifold, searching and ranking candidate trajectories without decoding to pixels (Sec. 3.3). Finally, we describe how **Hydra** operates as a local path follower when guided by an external global planner, via two different execution strategies (Sec. 3.4).

3.1 Problem Formulation: Navigation Paradigms

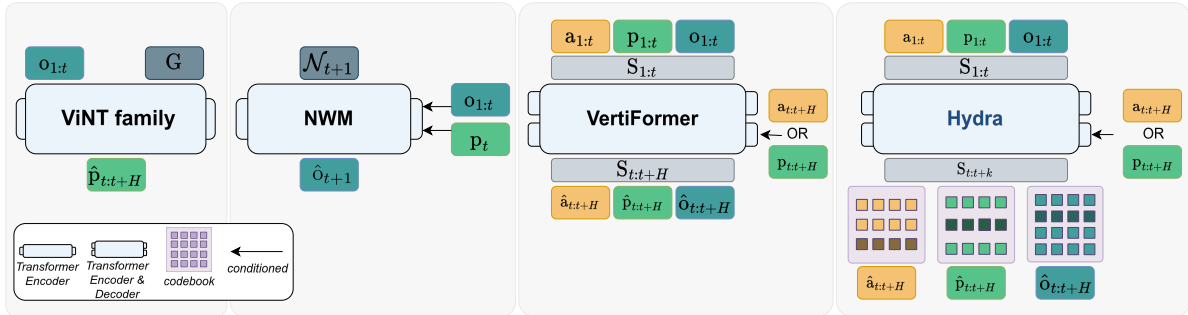


Figure 2: Comparison of **Hydra** architecture with Visual Navigation Transformer (ViNT), Navigation World Model (NWM), and VertFormer in terms of input modalities, conditioning, and the model output. **Hydra** builds on top of VertFormer and discretizes the output into intents to enable the model to search the discrete latent space for a plausible path.

We define autonomous navigation as the task of synthesizing an optimal sequence of actions to maximize a task-specific objective (e.g., reaching a goal) within a partially observable environment. At any discrete timestep t , the true state of the environment is unknown. Instead, the agent receives a multimodal observation tuple $O_t = \{I_t, p_t, a_{t-1}\}$, where $I_t \in \mathbb{R}^{3 \times H \times W}$ is the ego-centric visual observation, $p_t = (x_t, y_t, \theta_t)$ is the robot’s proprioceptive pose in a local odometric frame, and $a_{t-1} = (v, \omega)$ is the previously executed motor command.

The agent maintains a context history over a window K , defined as $\mathcal{H}_t = \{O_{t-K+1}, \dots, O_t\}$. Based on how models process this history to achieve goal-directed behavior, we categorize the current landscape of foundation navigation models into three distinct formulations.

Reactive Policy Formulation: Reactive foundation models (e.g., ViNT [58], GNM [57], and NoMaD [61]) frame navigation as a direct sequence-to-sequence mapping problem. Given the observation history $\mathcal{H}_t$ and a goal representation $\mathcal{G}$ (typically a target image or a directional vector planned by a human or an algorithm such as A^*), the model f_π

directly outputs a sequence of immediate future actions or spatial waypoints of length H :

$$\hat{a}_{t:t+H-1} = f_{\pi}(\mathcal{H}_t, \mathcal{G}). \quad (1)$$

While computationally lightweight and highly effective for local trajectory following, reactive policies lack generative foresight. They cannot predict the future visual or spatial consequences of their actions, making them highly susceptible to local minima and rendering them incapable of zero-shot constrained planning.

Visual World Model Formulation (Decoupled Planning): To overcome the myopia of reactive policies, WMs (*e.g.*, NWM [6]) frame navigation as a forward conditional dynamics task. Given the history $\mathcal{H}_t$ and a stochastically sampled sequence of future actions $a_{t:t+H-1}$ from a sampling-based planner, the model P_W generates the expected future visual states, $\hat{I}$:

$$\hat{I}_{t+1:t+H} \sim P_W(\cdot \mid \mathcal{H}_t, a_{t:t+H-1}). \quad (2)$$

This formulation enables Model Predictive Control (MPC) where the planner and WM operate on two separate manifolds. Because it strictly predicts visual states conditioned on continuous actions, it suffers from two critical flaws during deployment: it requires evaluating thousands of infeasible action sequences and relies on decoding the latent space to the actual modality, such as images, and comparing with the goal with brittle metrics such as perceptual metrics to extract spatial distance from generated images [1, 67, 8].

World Action Model Formulation (Unified Planning): To bridge the gap between generative foresight and kinodynamic reality, we formulate **Hydra** as a joint tri-modal WAM where the planner operates on the model’s manifold. Rather than isolating actions as inputs and images as outputs, **Hydra** models the joint generative flow of all three modalities simultaneously over a prediction horizon H . This joint modeling approach compensates for reactive policies’ inability to plan and WMs’ lack of spatial grounding and ignorance of physical and environmental constraints on control sequences. Meanwhile, the planner is inside the WM as opposed to the MPC approach of NWM, where the planner is an external module that calls the WM to evaluate a trajectory. This is because calculating the distance between two points on the manifold can tell us about the safety and physical plausibility of a path with zero decoding, and about progress toward the goal with only a single, cheap $O(1)$ decode of the winning candidate — never the expensive per-candidate image decoding NWM requires (see Fig. 1).

Given the observation history $\mathcal{H}_t$ and a sampled sequence of future actions $a_{t:t+H-1}$ or poses $p_{t+1:t+H}$, the model P_{WA} generates the expected future observations:

$$\hat{O}_{t+1:t+H} \sim P_{WA}(\cdot \mid \mathcal{H}_t, a_{t:t+H-1} \text{ or } p_{t+1:t+H}). \quad (3)$$

By predicting $\hat{p}$, **Hydra** enables precise geometric cost evaluation without relying on visual heuristics. By predicting $\hat{I}$, the model anticipates visual collisions. By predicting $\hat{a}$, the robot moves in the physical environment.

3.2 Hydra

In this section, we discuss the three primary mechanisms of **Hydra**: unified early-fusion encoding, disentangled discrete decoding, and multimodal latent flow matching. Figure 2 shows the evolution of **Hydra**’s architecture. In the Appendix, Table 7 compares **Hydra**’s modalities with existing approaches.

Unified Multimodal Encoding via Early Fusion. To resolve the dimensional disparity between vision and kinodynamics after patchifying without relying on computationally heavy cross-modal attention, **Hydra** adopts an early-fusion unified encoding strategy [44]. Visual latents $z_i \in \mathbb{R}^{D_z}$ are extracted via a frozen VAE [10]. Simultaneously, low-dimensional actions and poses undergo Fourier Feature Mapping [64, 86] to capture sharp control transitions. These modalities are broadcasted to a uniform length and summed to create a single, unified token stream [32]. To prevent receiving duplicated gradients for actions and poses due to broadcasting, which causes the model’s over-sensitivity to actions and poses, we rescaled actions and poses by their length, $\sqrt{k}$. To ensure robustness against partial observability during deployment (*e.g.*, missing telemetry) and to induce unconditioned exploration, we apply a learned modality masking strategy during training (details in Appendix C).

Furthermore, this unified history stream is concatenated with H learnable prediction tokens, where H is the forecasting horizon. The Transformer Encoder processes this joint sequence and outputs a fixed-size set of conditioning context tokens, $\text{ctx}_{t+1:t+1+H}$. These tokens efficiently compress the “where, what, and how” of the robot’s history.

The Discrete Bottleneck.

The Transformer Decoder cross-attends the context tokens $\text{ctx}_{t+1:t+1+H}$ with the intended future poses ($p_{t+1:t+1+H}$) or actions ($a_{t:t+H}$), utilizing the same missing-modality masks as the encoder. This yields H “intent” tokens (z_e), which are subsequently fed to modality-specific VQ codebooks [70, 31, 81], one per modality. Because each codebook

is a finite, enumerable set of candidate intents, the model’s uncertainty over which intent is appropriate reduces to a categorical distribution over a small vocabulary – a property that makes the predictive-entropy term of $\mathcal{J}_{KPC}$ (Sec. 3.3) a cheap computation, unlike a continuous action space where no such finite enumeration exists. To balance high-fidelity spatial forecasting with the strict latency constraints of closed-loop control, we employ *asymmetric vocabulary quantization*. We allocate a large codebook ($|\mathcal{V}_i| = 2048$) to the visual generative head, while strictly constraining the kinodynamic intents ($|\mathcal{V}_p| = 64, |\mathcal{V}_a| = 64$). This asymmetry preserves visual representational capacity while explicitly enabling the fast retrieval speeds required for DLP (see Appendix B for deterministic anchoring and codebook stabilization details).

Multimodal Generative Flow Matching.

For each modality, we define a probability path from a source Gaussian noise distribution $\psi_0 \sim \mathcal{N}(0, I)$ to the ground-truth target ψ_1 . At any continuous integration time $s \in [0, 1]$, the intermediate noisy state is $\psi_s = (1 - s)\psi_0 + s\psi_1$. Hydra employs three flow decoders [36] to predict the target velocity field for visuals, poses, and actions.

The stabilized, discrete context tokens and the time step s are fed into the Flow Decoders via DiT blocks [51]. The network is optimized end-to-end via the vector field loss for each modality:

$$\mathcal{L}_{flow} = \mathbb{E}_{s, \psi_0, \psi_1} \left[\|v_{\theta}^{(\psi)}(\psi_s, s, e_c) - (\psi_1 - \psi_0)\|_2^2 \right].$$

Please refer to Sec. B of the Appendix for more details about the design choices.

3.3 Planning in the Discrete Latent Manifold

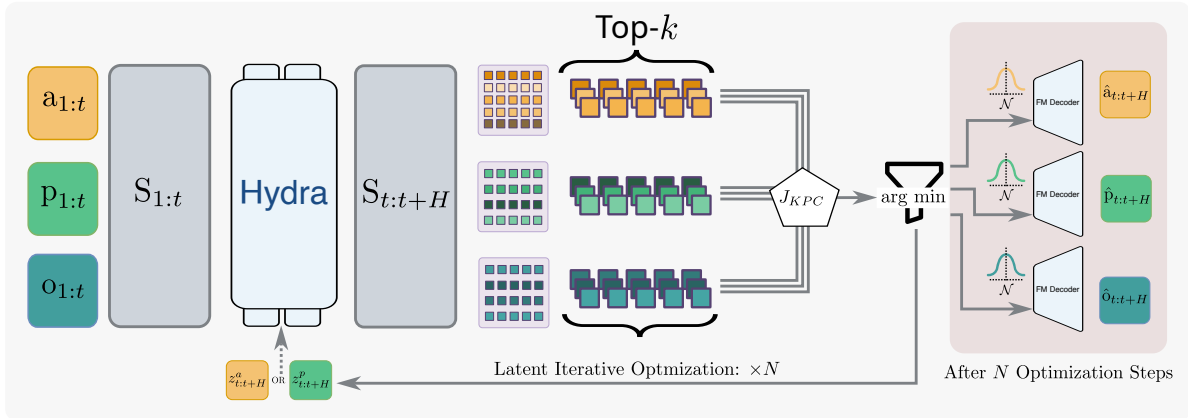


Figure 3: Initial intent tokens are generated unconditionally at first to become the seed ‘intent’. Then these future intents go through VQ codebooks, and we sample the top- k codes. After calculating the KPC cost, the lowest cost is chosen to seed the next iteration of searching. The best trajectory is decoded using flow matching heads to continuous robot poses and actions at the end for robot execution.

To bridge the gap between continuous planners, which frequently suffer from the computational waste of exploring bad trajectories, and world models that lack goal-directed control (Eq. (2)), Hydra introduces *Discrete Latent Planning* (DLP) with continuous FM execution (Eq. (3)). Instead of searching blindly in a dense continuous domain, Hydra operates explicitly over the learned topological manifold defined by the discrete codebooks $\mathcal{V}_i$, $\mathcal{V}_p$ and $\mathcal{V}_a$.

Unbiased Initialization and Gumbel-Max Search. In the absence of an explicit future intent, Hydra initializes its search using an *Unbiased Expert Prior* by querying the model with masked future states. Using this unbiased seed, Hydra generates a diverse batch of candidate topological paths by adding Gaussian noise. Furthermore, by applying exponentially smoothed Gumbel noise to the discrete codebook logits, we explore alternate topological routes while preserving kinodynamic smoothness across time steps.

Latent Optimization. Once discrete candidates are retrieved, they must be ranked. Previous visual planning models such as NWM, rely on decoding latents to pixels to compute perceptual similarity (e.g., LPIPS [82]), which incurs massive computational overhead. Furthermore, pure Euclidean pose distance introduces the “wall trap”: naively scoring a trajectory as optimal if it lies on the shortest linear path to the goal, regardless of physical occlusions. To solve this, Hydra evaluates candidates entirely within the discrete manifold using a multi-objective cost function, defined for a candidate intent sequence τ as:

$$\mathcal{J}_{\text{KPC}}(\tau) = \underbrace{\mathcal{C}_{\text{geo}}}_{\text{Geometric Goal}} + \sum_{t=1}^T \gamma^t \left(\underbrace{\lambda_{\text{img}} \mathcal{C}_{\text{img}}^{(t)}}_{\text{Visual Goal}} + \underbrace{\lambda_{\text{prior}} \mathcal{C}_{\text{prior}}^{(t)}}_{\text{Kinodynamic Prior}} + \underbrace{\lambda_{\text{ent}} \mathcal{C}_{\text{ent}}^{(t)} + \lambda_{\text{vq}} \mathcal{C}_{\text{vq}}^{(t)}}_{\text{Perceptual Safety}} \right) \quad (4)$$

where $\gamma \in (0, 1)$ is a temporal discount factor that heavily penalizes immediate infractions while decaying the weight of distant future states to account for compounding generative uncertainty. The terms operate synergistically to ensure safe execution:

- **Geometric and Semantic Tracking ($\mathcal{C}_{\text{geo}}, \mathcal{C}_{\text{img}}$):** Evaluates the L_2 deviation from the local goal coordinates and the MSE against the visual goal latent embedding. A direct path minimizes this cost, but requires regularization to survive occlusions.
- **Unbiased Expert Prior ($\mathcal{C}_{\text{prior}}$):** Computed as the Negative Log-Likelihood (NLL) of the sampled tokens against the unconditional predictive prior, an implicit measure of what a human demonstrator would plausibly do in this situation. Because the prior is fit only to collision-free human demonstrations, a trajectory that drives toward an obstacle is itself far outside this distribution; $\mathcal{C}_{\text{prior}}$ therefore contributes measurably to obstacle avoidance (Table 6).
- **Visual Predictive Entropy ($\mathcal{C}_{\text{ent}}$):** Computed as the Shannon entropy of the predicted visual token distribution. This term captures the model’s predictive uncertainty over the future visual state, which rises in ambiguous or partially observed regions of the scene, including occlusions. By penalizing high-entropy rollouts, $\mathcal{C}_{\text{ent}}$ contributes to trajectory safety, discouraging the planner from committing to futures the model cannot reliably predict.
- **Manifold Rejection ($\mathcal{C}_{\text{vq}}$):** Computed as the quantization error (cosine distance) between the continuous sampled intent and its nearest entry in the visual codebook $\mathcal{V}_i$, entirely in embedding space. If a candidate trajectory drives directly into a visual occlusion, the Transformer Decoder’s raw predicted embedding for that step has no close match in $\mathcal{V}_i$, *i.e.*, an unobserved physical state has no learned code to snap to. Because this state is OOD, it is thrown off the learned topological manifold of safe navigation, resulting in a manifold rejection spike, without ever invoking the Flow Matching decoder.

Together, $\mathcal{C}_{\text{prior}}$, $\mathcal{C}_{\text{ent}}$ and $\mathcal{C}_{\text{vq}}$ jointly contribute to obstacle avoidance and overall trajectory safety (Table 6): $\mathcal{C}_{\text{prior}}$ is safety implicitly encoded in demonstrations, $\mathcal{C}_{\text{ent}}$ flags predictive uncertainty over the future scene, and $\mathcal{C}_{\text{vq}}$ flags sampled intents that are directly infeasible under the learned dynamics manifold. Note that all KPC terms are calculated in the latent space except $\mathcal{C}_{\text{geo}}$, which must be decoded to compute the Euclidean distance to the goal, a negligible computational cost.

Once the elite trajectory minimizing $\mathcal{J}_{\text{KPC}}(\tau)$ is identified in the latent space, its discrete tokens are decoded into local spatial deltas. To mitigate computational latency and ensure smooth closed-loop execution, these local predictions are fused into the global physical execution path using Action Chunking with Temporal Ensembling [83], continuously blending overlapping horizon predictions as the robot navigates.

Empirically, when the continuous generative head is forced to evaluate a trajectory intersecting an occlusion, it frequently attempts to resolve the OOD state by hallucinating navigable free space (*e.g.*, morphing a physical obstacle into a corridor). The Manifold Rejection ($\mathcal{C}_{\text{vq}}$) and Predictive Entropy ($\mathcal{C}_{\text{ent}}$) costs explicitly detect and heavily penalize the predictive uncertainty of these reality-warping hallucinations, rejecting the trajectory before execution.

Real-Time Inference and Computational Pruning. By evaluating $\mathcal{J}_{\text{KPC}}(\tau)$ directly on the intermediate logits and intent tokens, Hydra bypasses the massive latency of decoding predicted observations back into pixel space. Because the heavy observation history is fully compressed into context tokens during the initial encoder pass, iterative refinement loops evaluate strictly through the transformer decoder, entirely in embedding space. Flow matching is called once to decode only the elite trajectory after search concludes ($N = 10$ NFEs for the kinodynamic decoders; $N = 50$ NFEs for the visual decoder). This fully latent search drives Hydra’s $> 500\times$ reduction in planning time relative to NWM, which must decode and score each candidate individually (Table 2). This is critical to enable Hydra to meet the strict real-time control loop constraints required for physical edge deployment.

3.4 Path Following via Global Guidance

While the DLP allows Hydra to act as a standalone global planner, state-of-the-art long-horizon navigation often relies on a planned path (*e.g.*, ViNT [58] and NoMaD [61]). In this mode, a global planner (such as A* on a topological graph or even a human-made path) provides a local subgoal g , and the generative model acts as a local reactive policy (Eq. (1)).

Under this paradigm, the mathematical objective fundamentally shifts. Rather than minimizing a cost function, **Hydra** performs goal-conditioned Behavior Cloning (BC). The objective is to find a latent intent sequence τ that minimizes the deviation from the provided geometric waypoint g , while remaining strictly bound to the safe, obstacle-free manifold learned during training. To do so, we propose two execution strategies that share a single *candidate-generation* mechanism and differ only in how they score the external guidance signal.

Classifier-Free Guidance in Kinodynamic Space. To blend structural safety with goal-directed tracking, we adapt Classifier-Free Guidance (CFG) [23] into the kinodynamic latent space. We query the transformer decoder twice. First, we mask the future pose and action conditions to compute the unconditional prior $z_\emptyset$, which represents the model’s natural, obstacle-avoiding intent based purely on the visual context. Second, we unmask the condition to compute the rigid geometric intent z_g requested by the global planner. We then extrapolate a new execution intent via linear interpolation:

$$\tilde{z}_g = z_\emptyset + \omega \cdot (z_g - z_\emptyset), \quad (5)$$

where ω is the guidance scale. This approach is efficient. However, because $\tilde{z}_g$ is computed via vector addition, it introduces the risk of *manifold deviation*. If a high ω forces the robot to track an A* path that directly intersects an unmapped dynamic obstacle, the linear extrapolation can push the latent vector outside the bounds of the discrete VQ codebook ($\mathcal{V}_p$), causing the downstream decoder to hallucinate physically impossible dynamics. To prevent this, as an alternative, we can perform interpolation on the decoded trajectories. The details of this approach are provided in App. E.2.

Sub-Goal Sampling. We also propose a sampling approach for path following. When guidance arrives as a single local lookahead subgoal g , we evaluate the decoded candidate trajectories against g and select the trajectory minimizing the geometric tracking cost:

$$\mathcal{J}_{\text{track}}(\tau) = \|\tau_T - g\|_2 + \frac{1}{2T} \sum_{t=1}^T \|\tau_t - g\|_2. \quad (6)$$

By generating candidates exclusively from the expert prior and the codebook vocabulary, with the goal condition masked during generation, every evaluated trajectory is a plausible maneuver under the learned dynamics, and local obstacle avoidance is inherited from the expert prior. More details of the sampling process are provided in App. E.3.

Computational Trade-offs. By restricting candidate generation exclusively to the unbiased expert prior (masking the goal condition during generation), every sampled trajectory lies on the learned manifold. While evaluating visual predictive entropy or manifold rejection over these candidates (as done in Sec. 3.3) would provide obstacle avoidance, we explicitly omit the image codebook from this local sampling loop. This omission is a strategic pruning choice designed to maintain low inference latency, relying entirely on the inherent safety of the expert before handling local obstacle avoidance. The primary limitation of this method occurs in highly constrained environments (*e.g.*, sharp U-turns), where the unconditional prior may assign near-zero probability to the necessary maneuver.

4 Experiments

The empirical evaluation of **Hydra** is designed to answer three core questions.

1. Generative Grounding and Temporal Consistency: Does **Hydra**’s multi-task, parameter-efficient architecture maintain competitive visual predictive fidelity compared to continuous baselines like NWM? Specifically, we evaluate whether our early fusion degrades or improves long-horizon video generation.

2. Real-Time Planning Efficiency: Does restricting the trajectory search space to a discrete intent manifold bypass the severe computational bottlenecks of continuous sampling, thereby enabling real-time edge deployment?

3. Policy Accuracy in Path Following Navigation: When deployed as a local path follower, how does **Hydra** compare to state-of-the-art reactive behavior cloning models (*e.g.*, ViNT, NoMaD, and GNM)? This evaluates whether our multimodal generative WAM does not sacrifice the strict observation-to-action behavior.

Comprehensive ablations validating discrete manifold stability during training (*e.g.*, deterministic anchoring) are provided in Appendix 4.5.

4.1 Generative Grounding: Long-Horizon Temporal Consistency

To evaluate whether Hydra’s early fusion and discrete bottleneck preserve temporal grounding and video generation quality, following NWM, we autoregressively generate 100 video clips of 16 seconds (at 4 FPS) from a held-out subset of the RECON dataset [56], ensuring the data is the same for all models. We benchmark against the base variant of NWM (NWM/B, 200M parameters, continuous) and VertiFormer (a deterministic baseline). All models are evaluated using *teacher-forcing* (ground-truth actions at each step) for a fair comparison of the visual flow solvers alone; unlike NWM, which *requires* these external actions to roll out, Hydra is a complete World Action Model fully capable of generating trajectories via its own endogenous action head.

As shown in Table 1, Hydra outperforms NWM in Fréchet Video Distance (FVD), which measures temporal coherence and action adherence. NWM’s continuous latent space suffers from compounding errors and visibly decouples from the commanded actions after approximately 8 seconds (Appendix A.2), while Hydra’s multimodal manifold enforces strict temporal grounding across the full 16-second horizon, despite the inherent quantization limits of its discrete visual codebook ($|\mathcal{V}_i| = 2048$). Single-step predictive fidelity (DreamSim, LPIPS, PSNR) is reported in Appendix A.1.

4.2 Planning: Real-Time Feasibility via Discrete Search

While recent literature frequently benchmarks generative planners using static offline datasets, we argue that this methodology is fundamentally misaligned with the objectives of autonomous planning. Offline datasets strictly measure reactive Behavior Cloning; they penalize any topological deviation from the human demonstration (τ_{GT}). If a search-based planner discovers an alternative, physically valid, and collision-free trajectory τ , static evaluations incorrectly compute a high geometric penalty $L = \|\tau - \tau_{GT}\|^2$. True exploratory planning and obstacle avoidance must therefore be evaluated through closed-loop physical deployment.

Table 1: Comparison of Video Synthesis Quality. 100 clips of 16-seconds generated at 4 FPS. Hydra outperforms baselines in temporal coherence (FVD).

Model	FVD ↓	Time (100 clips)
NWM/B	22.1087 ± 10.7199	3.5h
VertiFormer	81.0325 ± 12.5069	2m
Hydra	19.1588 ± 5.1180	4m

To evaluate the computational efficiency and goal-directed planning capabilities of our DLP, we deploy Hydra on two robot embodiments: Clearpath Jackal and Boston Dynamics Spot. Because unmapped, long-horizon planning remains a fundamental open challenge, our evaluation focuses strictly on close-range navigation: the robot must autonomously plan a collision-free trajectory to a destination approximately 8 meters away, initialized in 5 variable starting orientations ($\pm 40^\circ$) across both unobstructed and partially occluded environments. A trial is deemed successful if the robot navigates within 0.5 meters of the goal in under 3 minutes without collision. To ensure a rigorous, isolated comparison of planning latency, all physical inference is offloaded to a dedicated workstation (NVIDIA A5000, 24GB VRAM), a high-compute configuration adopted out of strict necessity to accommodate NWM’s continuous sampling paradigm which is entirely unsuited for edge hardware, and in our physical trials, attempting to execute the NWM’s planning loop on a standard mobile computing unit consistently induced control-loop timeouts and systemic failure.

We benchmark Hydra against VertiFormer [44] and the base variant of NWM (NWM/B); we omit NWM/S as its reduced parameter count degrades predictive fidelity without resolving the fundamental latency bottleneck caused by iterative denoising steps combined with CEM sampling over LPIPS image cost function ($\mathcal{C}_{img}$). VertiFormer serves as a continuous ablation of our discrete architecture. While it shares Hydra’s early-fusion design, its reliance on an unbounded continuous state space forces it to fall back on computationally heavy MPPI sampling. Crucially, without a discrete latent representation, VertiFormer cannot leverage our prior, entropy, and quantization cost terms ($\mathcal{C}_{prior}$, $\mathcal{C}_{ent}$, and $\mathcal{C}_{vq}$), restricting its cost function to less informative geometric and image-space terms ($\mathcal{C}_{geo}$ and $\mathcal{C}_{img}$). Consequently, both baselines suffer from severe sampling bottlenecks and degraded optimization landscapes that Hydra’s discrete codebooks eliminate.

Table 2: Planning Performance. While continuous VertiFormer with MPPI runs quickly, it suffers from sample inefficiency and being blind to obstacles (0% SR). Hydra’s DLP achieves high success rates while maintaining real-time execution speeds compared to NWM.

Model	Params	Search Space	Iterations	Samples	Len.	Plan Time ↓	SR (Unobs.) ↑	SR (Obs.) ↑	Corner Turn ↑
NWM/B	200M	Continuous (CEM)	3	18	12	> 500.0s	0/10	0/10	0/10
VertiFormer	27.11M	Continuous (MPPI)	1	18	12	~ 0.7s	4/10	0/10	0/10
Hydra	143.29M	Discrete (DLP)	3	18	12	~ 0.9s	10/10	8/10	8/10

The Failure of Continuous Search Spaces. Continuous WMs require optimization over unbounded action spaces, leading to distinct failure modes. As shown in Table 2, when evaluating identical hyperparameters (3 iterations, 18 samples, length 12), NWM suffered from catastrophic latency: one CEM search required over 500 seconds, rendering it physically undeployable. The continuous VertiFormer baseline uses MPPI with only one planning iteration in ~ 0.7 seconds, but suffers from *sample inefficiency*: without a structural prior bounding the search space, continuous MPPI generates trajectories that cause collisions, like going through an obstacle, resulting in a 0% Success Rate across scenarios with obstacles.

Real-Time DLP. By restricting the search space to a discrete manifold of kinodynamic primitives directly in latent space without decoding, Hydra plans in ~ 0.9 seconds—a $\sim 500\times$ speedup over NWM. More importantly, this discrete structural prior yields physically feasible trajectories: Hydra achieves a perfect 10/10 success rate in unobstructed environments and 8/10 in both occluded and blind-corner scenarios. These results empirically demonstrate that latent space discretization, and the resulting inclusion of $\mathcal{C}_{\text{prior}}$, $\mathcal{C}_{\text{ent}}$, and $\mathcal{C}_{\text{vq}}$, improves planning safety across diverse topological challenges, a capability inherently beyond continuous-space models like VertiFormer.

4.3 Latent Collision Detection

To empirically validate Manifold Rejection ($\mathcal{C}_{\text{vq}}$) and Predictive Entropy ($\mathcal{C}_{\text{ent}}$) as complementary latent collision detectors, we visualize both signals jointly for Hydra when forced into a geometric contradiction: a trajectory commanded to drive straight into an obstacle.

We compute the VQ quantization error of the visual tokens and the Shannon entropy of their distribution at each rollout step. As the commanded trajectory approaches the obstacle, the continuous intent vector drifts toward the undefined space between valid discrete codebook anchors, and both signals rise in tandem: quantization error rises because the predicted embedding has no close codebook match, and entropy rises because the model becomes less confident about which future is plausible. When the generative head is ultimately forced to resolve this OOD state – a *Contextual Manifold Shift* – by hallucinating a navigable pavement in place of the occlusion, both signals drop together immediately afterward, because the fabricated resolution is once again close to something the model has seen and is confident about.

The signal is not perfectly discriminating, however. The smaller fluctuation in $\mathcal{C}_{\text{vq}}$ visible earlier in the rollout is a false positive: the model registers mildly elevated quantization error while approaching visually complex but entirely traversable foliage, rather than a genuine obstacle. This introduces a limitation – dense, textured content such as leaves and grass can locally resemble the reconstruction difficulty of a true occlusion, even though no collision risk is present. Hydra’s collision detection is therefore a strong, but not infallible, signal.

This joint behavior is nonetheless the crux of Hydra’s DLP. Because $\mathcal{C}_{\text{vq}}$ and $\mathcal{C}_{\text{ent}}$ are computed independently – one from quantization geometry, one from predictive uncertainty – but rise and fall together at exactly the moments that matter, heavily weighting both in the KPC (Eq. (4)) lets the optimizer reject a candidate trajectory *before* the generative head is forced to hallucinate, using two agreeing, pixel-free signals rather than one. Occasional false positives aside, this remains substantially more reliable than the alternative of not evaluating safety at all until pixels are rendered: the entire mechanism – detection, ranking, and rejection – happens in latent space, extracting a safety constraint directly from the quantization and uncertainty topology of the visual world model without ever decoding to pixels.

4.4 Local Path Following: Hydra vs. Reactive Policies

We next evaluate Hydra as a *local path follower* that tracks waypoints from an external guidance signal (Sec. 3.4), and directly compare this against Hydra operating in its own planning mode (Sec. 4.2), under the same physical conditions.

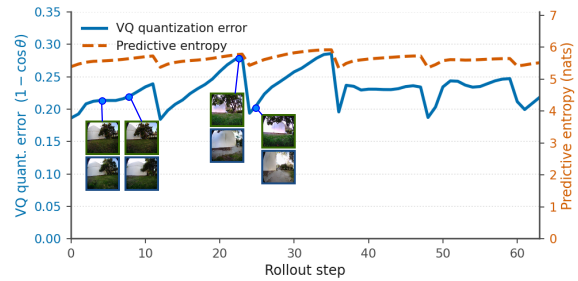


Figure 4: **Manifold Rejection and Predictive Entropy During Impending Collision.** Top/Bottom insets: Comparison of Ground Truth vs. Hydra’s imagination at points along a rollout commanded to drive straight toward a physical occlusion. Both the spatially-averaged VQ quantization error ($\mathcal{C}_{\text{vq}}$, solid blue, left axis) and the predictive entropy of the visual token distribution ($\mathcal{C}_{\text{ent}}$, dashed orange, right axis) rise together as the commanded trajectory approaches the occlusion, and drop together immediately after the model hallucinates a navigable resolution – two independently computed signals converging on the same collision-relevant moments. The smaller early fluctuation (first two insets) is a false positive triggered by visually complex but traversable foliage, not a genuine obstacle.

Table 3: Local Planning Evaluation across 10 Physical Trials per Scenario. **Hydra** drastically reduces physical interventions by leveraging its predictive foresight to move around topological traps that ensnare reactive baselines. GNM, ViNT, and NoMaD are conditioned on dense goal-image sequences at 2Hz; **Hydra**’s local-following variants (CFG, Sampling) are conditioned on sparse, more widely spaced waypoints.

Scenario	Metric	Dense Goal Images (2Hz)			Sparse Waypoints (turns only)			
		GNM	ViNT	NoMaD	Hydra (CFG)	Hydra (Sampling)	VertiFormer	Hydra (Planning)
Obstructed Goal	Success Rate (%) ↑	90	80	100	90	50	100	90
	Traversal Time (s) ↓	36.4 ± 7.9	34.4 ± 6.8	37.4 ± 7.7	45.0 ± 8.0	48.4 ± 18.5	21.0 ± 1.05	37.2 ± 21.0
	Interventions ↓	1.1 ± 0.3	1.2 ± 0.6	1.1 ± 0.3	1.0 ± 0.7	0.8 ± 0.8	1.0 ± 0.5	1.2 ± 0.6
Long Distance	Success Rate (%) ↑	0	0	0	100	100	100	40
	Traversal Time (s) ↓	65.8 ± 3.7	46.0 ± 7.1	91.8 ± 22.2	138 ± 7.7	107 ± 8.6	109 ± 7.6	119 ± 3.8
	Interventions ↓	0.8 ± 0.8	2.0 ± 2.0	0.4 ± 0.5	2.4 ± 0.9	1.2 ± 0.8	1.0 ± 1.0	3.4 ± 0.6
Indoor Corridor	Success Rate (%) ↑	40	90	90	100	100	100	100
	Traversal Time (s) ↓	59.0 ± 4.6	48.1 ± 3.3	65.0 ± 21.2	24.4 ± 2.01	28.2 ± 0.79	26.3 ± 1.6	33.9 ± 2.0
	Interventions ↓	0.0 ± 0.0	0.0 ± 0.0	0.2 ± 0.4	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.1 ± 0.3

Table 3 reports Success Rate, Traversal Time, and Interventions on Spot across three scenarios – Obstructed Goal, Long Distance, and Indoor Corridor – for three reactive baselines (GNM, ViNT, and NoMaD), VertiFormer, **Hydra**’s two local-following variants (CFG and Sampling), and **Hydra**’s planner.

Guidance density differs substantially across methods. GNM, ViNT, and NoMaD are conditioned on a dense stream of goal images sampled at 2Hz. **Hydra**’s local-following variants, by contrast, are conditioned on sparse waypoints that may be spaced substantially farther apart in both time and space. Despite receiving far less frequent correction, **Hydra** matches or exceeds the reactive baselines’ success rate and intervention counts in every scenario tested.

CFG and Sampling trade off differently across horizons. **Hydra**’s two local-following variants are complementary rather than uniformly ranked. CFG achieves the lowest traversal time and zero interventions on Indoor Corridor and is competitive on Obstructed Goal, but degrades sharply on Long Distance (138 ± 7.7 s, 2.4 ± 0.9 interventions), underperforming even the reactive baselines on traversal time. This is consistent with the manifold-deviation risk noted in Sec. 3.4: because $\tilde{z}_g$ is computed via continuous interpolation between z_g and $z_\emptyset$, small per-step deviations compound over the many timesteps a long-distance trajectory requires, eventually pushing the latent outside the valid VQ manifold and triggering hallucinated dynamics – an effect negligible over the short horizons of Obstructed Goal and Indoor Corridor but substantial over Long Distance. Sampling, which restricts every candidate to the valid codebook by construction rather than extrapolating between samples, does not exhibit this failure mode and is the more reliable choice for long-horizon following, at some cost in traversal time on the shorter scenarios. This indicates that **Hydra**’s world model does not degrade the policy’s performance, but rather improves it.

VertiFormer remains a strong baseline on raw speed, achieving the fastest traversal time on Obstructed Goal (21.0 ± 1.05 s) and matching the best intervention count on Indoor Corridor. This is consistent with a deterministic regression head reaching a single hypothesis directly, without the search or multi-step generative rollout that **Hydra**’s planning, CFG, and Sampling variants perform, trading the collision-avoidance and goal-flexibility benefits (Table 6) for raw speed. For completeness, Table 4 reports offline trajectory forecasting accuracy (ADE/FDE) across four navigation datasets (SCAND, TartanDrive, SACSoN, and RECON); as here, deterministic baselines such as VertiFormer edge out generative methods on this metric.

Planning vs. local following. **Hydra**’s planning mode is included to compare directly against its own local-following variants under matched conditions with an even sparser external waypoint signal. On Obstructed Goal, planning reaches a 90% success rate with 37.2 ± 21.0 s traversal time and 1.2 ± 0.6 interventions – matching CFG’s success rate but slower and with substantially higher variance, consistent with performing search at every replanning step rather than following a supplied waypoint. On Indoor Corridor, planning matches the 100% success rate achieved by every **Hydra** variant, with only 0.1 ± 0.3 interventions, trailing CFG and Sampling’s traversal time by roughly 6–10 seconds.

On Long Distance, by contrast, planning succeeds in only 40% of trials, with a substantially higher traversal time (119s) and intervention count (3.4 ± 0.6) than every other

Table 4: Offline trajectory forecasting evaluation, averaged across four navigation datasets (SCAND, TartanDrive, SACSoN, and RECON). **Hydra** achieves competitive policy accuracy while retaining the planning capability that reactive baselines lack.

Model	Params	ADE ↓	FDE ↓
GNM	8.6M	0.32	0.53
ViNT	29.6M	0.37	0.61
NoMaD	19.0M	0.59	1.06
VertiFormer	27.11M	0.21	0.39
Hydra	143.29M	0.24	0.42

Table 6: Ablation of obstacle-avoidance cost terms (λ_{prior} , λ_{vq} , λ_{entropy}), across two robot embodiments: a blind corner turn (Spot) and frontal obstacle avoidance (Jackal). Each avoidance term degrades performance when removed on *both* platforms, indicating the mechanism is embodiment-agnostic.

Configuration	Spot		Jackal	
	Time (s) ↓	Collisions ↓	Time (s) ↓	Collisions ↓
Original (avoidance costs + $\mathcal{C}_{\text{geo}}$)	30.6 ± 12.1	0.20 ± 0.42	23.8 ± 0.39	0.20 ± 0.13
$\lambda_{\text{prior}} = 0$	33.9 ± 13.0	0.70 ± 0.48	21.2 ± 0.26	0.70 ± 0.59
$\lambda_{\text{vq}} = 0$	30.3 ± 10.0	0.50 ± 0.53	20.9 ± 0.34	0.60 ± 0.28
$\lambda_{\text{entropy}} = 0$	36.2 ± 18.9	0.40 ± 0.52	26.6 ± 0.45	0.80 ± 0.61

Hydra variant. Iterative search at every step is slower per decision than CFG’s interpolation or Sampling’s single forward pass, and this added latency is enough to make the robot react late at the scenario’s single corridor fork. In the shorter scenarios, a late reaction rarely matters, but Long Distance offers no second chance: committing to the wrong branch at the fork is unrecoverable within the trial’s time limit, whereas CFG and Sampling react quickly enough to take the correct branch consistently. This is a latency-budget mismatch rather than a failure of the discrete search itself – planning trades decision speed for operating without any external guidance, which is a poor trade specifically at irrecoverable decision points, and a good one everywhere else this evaluation tests it. We report it here because the same trade explains why CFG and Sampling exist as separate execution modes in the first place, not merely as a limitation to disclose. More details are provided in App. A.5.

4.5 Ablations

We ablate the individual terms of $\mathcal{J}_{\text{KPC}}$ (Eq. 4) to isolate which components drive **Hydra**’s obstacle-avoidance behavior. All results are averaged over 10 physical trials per configuration.

Goal visibility gates the image-goal term. We first ask whether $\mathcal{C}_{\text{img}}$ should be active unconditionally. Table 5 compares two scenarios that differ only in whether the goal is visible from the ego camera: frontal obstacle avoidance, where the goal remains at least partially in view, and a blind corner turn, where the goal is entirely outside the camera view until after the turn. With $\mathcal{C}_{\text{img}}$ active in both cases, success rate drops from 80% when the goal is visible to 20% when it is not. Without a true visual anchor for the goal, the term can be minimized by any visually plausible candidate rather than the geometrically correct one, actively misleading the planner rather than aiding it. Based on this result, the subsequent experiment, $\mathcal{C}_{\text{img}}$ is disabled.

Table 5: Effect of the image-goal cost term ($\lambda_{\text{img}} > 0$) conditioned on goal visibility. The same term that improves performance when the goal is visible degrades it when the goal is not, motivating conditioning $\mathcal{C}_{\text{img}}$ on goal visibility rather than applying it unconditionally.

Scenario	SR with $\mathcal{C}_{\text{img}}$ (%) ↑	Time (s) ↓
Goal visible (Frontal)	80	24.7 ± 5.8
Goal not visible (Blind corner)	20	30.6 ± 12.1

The remaining avoidance terms are individually necessary and embodiment-agnostic. With $\mathcal{C}_{\text{img}}$ handled as above, Table 6 ablates the three remaining terms – $\mathcal{C}_{\text{prior}}$, $\mathcal{C}_{\text{vq}}$, and $\mathcal{C}_{\text{ent}}$ – individually, on both Spot and Jackal. Removing any single term increases collisions relative to the full cost on both platforms, with the effect even more pronounced on Jackal (0.20 rising as high as 0.80) than on Spot (0.20 rising to 0.40–0.70). That the same qualitative degradation appears on two robots with different kinodynamics indicates each term contributes a distinct, non-redundant safety signal.

Notably, $\mathcal{C}_{\text{prior}}$ ’s contribution follows from the unconditional expert prior being fit only to collision-free human demonstrations: a trajectory that intersects an obstacle is itself far outside this distribution. $\mathcal{C}_{\text{prior}}$ therefore penalizes collisions as a direct consequence of penalizing implausibility under the expert distribution, complementing $\mathcal{C}_{\text{vq}}$ ’s explicit OOD rejection in the visual codebook rather than duplicating it.

5 Limitations and Future Work

While **Hydra** demonstrates advancements in latent planning, empirical deployment reveals fundamental challenges in how the discrete representation itself limits downstream navigation behavior.

Manifold Rejection Confuses Visual Complexity with Genuine Risk. Manifold Rejection ($\mathcal{C}_{vq}$) is a useful but imperfect safety signal: it cannot fully distinguish a genuinely dangerous occlusion from a visually complex but entirely safe scene. As shown empirically in Fig. 4, dense, high-frequency textures such as foliage and grass produce quantization error nearly as large as a true physical obstacle, since both are difficult for the visual codebook to reconstruct precisely, one because it is genuinely unfamiliar and dangerous, the other simply because it is visually intricate. The reverse failure is also possible: homogeneous obstacles such as flat walls are comparatively easy to reconstruct and may therefore be under-flagged despite being genuine hazards. Because the present signal cannot separate *why* a state is hard to reconstruct, occasional false positives and potentially false negatives are unavoidable with the current architecture. Addressing this will likely require a representation that distinguishes structural novelty that should be avoided from complex visual texture that is safe to traverse, rather than collapsing both into a single quantization-error signal.

Semantic Omission and Agent Erasure. Hydra exhibits persistent challenges in representing dynamic agents, particularly pedestrians. In complex social scenarios, these agents are frequently either erased or smoothed into the background. We hypothesize this limitation is twofold: First, the limited cardinality of our visual codebook ($|\mathcal{V}_i| = 2048$) acts as a stochastic low-pass filter, where humans, occupying a small subset of pixels, are discarded as high-frequency noise during quantization. Second, we inherit the spectral bias of the underlying Stable Diffusion VAE, which prioritizes global structure over small-scale semantic entities. Consequently, Hydra is better suited for static-world navigation; future work can benefit from incorporating human-centric object detection to ensure safe navigation around dynamic participants.

Representational Capacity Trades Expressivity for Latency. Hydra’s zero-shot generalization is intrinsically tied to the vocabulary size of its VQ codebooks. A larger codebook would capture a wider distribution of visual and kinodynamic behaviors, improving generalization to novel environments, but at a direct computational cost: retrieving the nearest codebook entry during search requires a distance computation whose cost scales with vocabulary size, contradicting the same real-time latency budget that motivated discretization in the first place (Sec. 4.2). Hydra therefore faces a structural trade-off between behavioral expressivity and search latency that a fixed-size codebook cannot resolve on its own; adaptive or hierarchical vocabularies are a natural direction for future work.

6 Conclusion

In this work, we introduced Hydra, a multimodal WAM that bridges the gap between generative planning and real-time physical execution for robotic navigation. By unifying visual states, poses, and actions through an early-fusion encoder, and forcing the generative process through a Vector-Quantized bottleneck, Hydra restricts the planning search space strictly to learned kinodynamic primitives. This structural prior introduces DLP. By leveraging our novel KPC, Hydra evaluates safety and search almost entirely in latent space, without the pixel-decoding bottleneck that limits continuous baselines. Extensive physical deployments demonstrate that Hydra outperforms continuous baselines (*e.g.*, NWM and VertFormer) in real-time goal-directed planning, at a scale and latency compatible with eventual onboard deployment. Ultimately, Hydra proves that discretizing a robot’s intent is a highly effective paradigm for scaling world models to the physical world – provided, as our limitations show, that the representation is expressive enough to capture what the codebook currently omits. Future work will pursue the directions raised above, adaptive codebook capacity and explicit dynamic-agent representation.

Acknowledgment

This work is a collaboration between the RobotiXX Lab, the VCAI Lab, and the AIR Lab. Research at the RobotiXX Laboratory is supported by the National Science Foundation (NSF, 2350352), the Army Research Office (ARO, W911NF2320004, W911NF2520011), the Army Ground Vehicle Systems Center (GVSC), Google DeepMind (GDM), Microsoft Research (MSR), Clearpath Robotics, FrodoBots Lab, Raytheon Technologies (RTX), Tangenta, 4-VA, the Mason Innovation Exchange (MIX), and Walmart. Research by the VCAI Lab was conducted under the RobOdin project, funded by the European Union through its Interreg Germany-Denmark funding program, with technical support provided by the Embodied AI Center at Kiel University. Work at the AIR Lab was supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) and Information Technology Research Center (ITRC) grant, funded by the Korean government (Ministry of Science and ICT) (IITP-2026-RS-2020-II201460).

References

- [1] Bo Ai, Stephen Tian, Haochen Shi, Yixuan Wang, Tobias Pfaff, Cheston Tan, Henrik I Christensen, Hao Su, Jiajun Wu, and Yunzhu Li. A review of learning-based dynamics models for robotic manipulation. *Science Robotics*, 10(106):eadt1497, 2025.

- [2] Arslan Ali, Junjie Bai, Maciej Bala, Yogesh Balaji, Aaron Blakeman, Tiffany Cai, Jiaxin Cao, Tianshi Cao, Elizabeth Cha, Yu-Wei Chao, et al. World simulation with video foundation models for physical ai. *arXiv preprint arXiv:2511.00062*, 2025.
- [3] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture, April 2023.
- [4] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [5] Randall Balestrierio and Yann LeCun. LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics, November 2025.
- [6] Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 15791–15801, 2025.
- [7] Adrien Bardes, Jean Ponce, and Yann LeCun. MC-JEPA: A Joint-Embedding Predictive Architecture for Self-Supervised Learning of Motion and Content Features, July 2023.
- [8] Daniel M Bear, Elias Wang, Damian Mrowca, Felix J Binder, Hsiao-Yu Fish Tung, RT Pramod, Cameron Holdaway, Sirui Tao, Kevin Smith, Fan-Yun Sun, et al. Physion: Evaluating physical prediction from vision in humans and machines. *arXiv preprint arXiv:2106.08261*, 2021.
- [9] Homanga Bharadhwaj, Debidatta Dwibedi, Abhinav Gupta, Shubham Tulsiani, Carl Doersch, Ted Xiao, Dhruv Shah, Fei Xia, Dorsa Sadigh, and Sean Kirmani. Gen2act: Human video generation in novel scenarios enables generalizable robot manipulation. In *Conference on Robot Learning*, pages 3936–3951. PMLR, 2025.
- [10] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.
- [11] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [12] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024.
- [13] Sam Buchanan, Druv Pai, Peng Wang, and Yi Ma. *Learning deep representations of data distributions*. Online, August 2025.
- [14] Faris Chaudhry. The geometry of projection heads: Conditioning, invariance, and collapse. *arXiv preprint arXiv:2605.17180*, 2026.
- [15] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning*, pages 1597–1607. PMLR, 2020.
- [16] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [17] Yilun Du, Sherry Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Josh Tenenbaum, Dale Schuurmans, and Pieter Abbeel. Learning universal policies via text-guided video generation. *Advances in neural information processing systems*, 36:9156–9172, 2023.
- [18] Kuan Fang, Patrick Yin, Ashvin Nair, Homer Rich Walke, Gengchen Yan, and Sergey Levine. Generalization with lossy affordances: Leveraging broad offline data for learning visuomotor tasks. In *Conference on Robot Learning*, pages 106–117. PMLR, 2023.
- [19] Kartik Gupta, Thalaiyasingam Ajanthan, Anton van den Hengel, and Stephen Gould. Understanding and improving the role of projection head in self-supervised learning. *arXiv preprint arXiv:2212.11491*, 2022.
- [20] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in neural information processing systems*, volume 31. Curran Associates, Inc., 2018.
- [21] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering Diverse Domains through World Models, January 2023.

- [22] Noriaki Hirose, Dhruv Shah, Ajay Sridhar, and Sergey Levine. SACSoN: Scalable Autonomous Data Collection for Social Navigation, June 2023.
- [23] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.
- [24] Emiel Hoogeboom, Alexey A. Gritsenko, Jasmijn Bastings, Ben Poole, Rianne van den Berg, and Tim Salimans. Autoregressive diffusion models. In *International Conference on Learning Representations*, 2022.
- [25] Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. GAIA-1: A Generative World Model for Autonomous Driving, September 2023.
- [26] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with Diffusion for Flexible Behavior Synthesis, May 2022.
- [27] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. *arXiv preprint arXiv:2110.09348*, 2021.
- [28] Haresh Karnan, Anirudh Nair, Xuesu Xiao, Garrett Warnell, Soeren Pirk, Alexander Toshev, Justin Hart, Joydeep Biswas, and Peter Stone. Socially Compliant Navigation Dataset (SCAND): A Large-Scale Dataset of Demonstrations for Social Navigation. *arXiv:2203.15041 [cs, eess]*, March 2022.
- [29] Rooholla Khorrambakht, Joaquim Ortiz-Haro, Joseph Amigo, Omar Mostafa, Daniel Dugas, Franziska Meier, and Ludovic Righetti. Worldplanner: Monte carlo tree search and mpc with action-conditioned visual world models. *arXiv preprint arXiv:2511.03077*, 2025.
- [30] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning, January 2026.
- [31] Doyup Lee, Chihoon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11523–11532, 2022.
- [32] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- [33] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The eleventh international conference on learning representations*, 2023.
- [34] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- [35] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [36] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow, September 2022.
- [37] Xinhao Liu, Jintong Li, Yicheng Jiang, Niranjana Sujay, Zhicheng Yang, Juexiao Zhang, John Abanes, Jing Zhang, and Chen Feng. Citywalker: Learning embodied urban navigation from web-scale videos. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 6875–6885, 2025.
- [38] Zhiheng Liu, Weiming Ren, Xiaoke Huang, Shoufa Chen, Tianhong Li, Mengzhao Chen, Yatai Ji, Sen He, Jonas Schult, Belinda Zeng, Tao Xiang, Wenhui Chen, Ping Luo, Luke Zettlemoyer, and Yuren Cong. Tuna-2: Pixel Embeddings Beat Vision Encoders for Multimodal Understanding and Generation, April 2026.
- [39] Pauline Luc, Camille Couprie, Yann LeCun, and Jakob Verbeek. Predicting future instance segmentation by forecasting convolutional features. In Vittorio Ferrari, Martial Hebert, Cristian Sminchisescu, and Yair Weiss, editors, *Computer Vision – ECCV 2018*. Springer International Publishing, 2018.
- [40] Pauline Luc, Natalia Neverova, Camille Couprie, Jakob Verbeek, and Yann LeCun. Predicting deeper into the future of semantic segmentation. In *2017 IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [41] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [42] Riccardo Mereu, Aidan Scannell, Yuxin Hou, Yi Zhao, Aditya Jitta, Antonio Dominguez, Luigi Acerbi, Amos Storkey, and Paul Chang. Generative world modelling for humanoids: 1x world model challenge technical report. *arXiv preprint arXiv:2510.07092*, 2025.
- [43] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are Sample-Efficient World Models, March 2023.

- [44] Mohammad Nazeri, Anuj Pokhrel, Alexandyr Card, Aniket Datar, Garrett Warnell, and Xuesu Xiao. Vertiformer: A data-efficient multi-task transformer for off-road robot mobility. *arXiv preprint arXiv:2502.00543*, 2025.
- [45] E-Ro Nguyen, Yichi Zhang, Kanchana Ranasinghe, Xiang Li, and Michael S Ryoo. Pixel motion diffusion is what we need for robot control. *arXiv preprint arXiv:2509.22652*, 2025.
- [46] NVIDIA. Cosmos 3: Omnimodal World Models for Physical AI, June 2026.
- [47] NVIDIA, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, Jiaojiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, Gergely Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixe, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezani, Fitsum Reda, Xiaowei Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne Tchammi, Przemek Tredak, Wei-Cheng Tseng, Jibin Varghese, Hao Wang, Haoxiang Wang, Heng Wang, Ting-Chun Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Lin Yen-Chen, Xiaohui Zeng, Yu Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zolkowski. Cosmos World Foundation Model Platform for Physical AI, January 2025.
- [48] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An Open-Source Generalist Robot Policy, May 2024.
- [49] Chaoyi Pan, Giri Anantharaman, Nai-Chieh Huang, Claire Jin, Daniel Pfrommer, Chenyang Yuan, Frank Permenter, Guannan Qu, Nicholas Boffi, Guanya Shi, and Max Simchowitz. Much ado about noising: Dispelling the myths of generative robotic control. 2025.
- [50] Jiwon Park, Dongkyu Lee, I. Made Aswin Nahrendra, Jaeyoung Lim, and Hyun Myung. DreamFlow: Local Navigation Beyond Observation via Conditional Flow Matching in the Latent Space, March 2026.
- [51] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.
- [52] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gómez Colmenarejo, Alexander Novikov, Gabriel Barthmaron, Mai Giménez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent. *Transactions on Machine Learning Research*, 2022.
- [53] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models. *arXiv:2112.10752 [cs]*, April 2022.
- [54] Reuven Y Rubinstein. Optimization of computer simulation models with rare events. *European Journal of Operational Research*, 99(1):89–112, 1997.
- [55] Lloyd Russell, Anthony Hu, Lorenzo Bertoni, George Fedoseev, Jamie Shotton, Elahe Arani, and Gianluca Corrado. Gaia-2: A controllable multi-view generative world model for autonomous driving. *arXiv preprint arXiv:2503.20523*, 2025.
- [56] Dhruv Shah, Benjamin Eysenbach, Nicholas Rhinehart, and Sergey Levine. Rapid exploration for open-world navigation with latent goal models. In *5th Annual Conference on Robot Learning*, 2021.
- [57] Dhruv Shah, Ajay Sridhar, Arjun Bhorkar, Noriaki Hirose, and Sergey Levine. GNM: A general navigation model to drive any robot. In *International Conference on Robotics and Automation (ICRA)*, 2023.
- [58] Dhruv Shah, Ajay Sridhar, Nitish Dashora, Kyle Stachowicz, Kevin Black, Noriaki Hirose, and Sergey Levine. ViNT: A Foundation Model for Visual Navigation. *arXiv*, June 2023.
- [59] Vlad Sobal, Wancong Zhang, Kynghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from Reward-Free Offline Data: A Case for Planning with Latent Dynamics Models, February 2025.
- [60] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics, November 2015.
- [61] Ajay Sridhar, Dhruv Shah, Catherine Glossop, and Sergey Levine. NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration, October 2023.
- [62] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. *arXiv preprint arXiv:2602.22010*, 2026.

- [63] Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. Vla-jepa: Enhancing vision-language-action model with latent world model. *arXiv preprint arXiv:2602.10098*, 2026.
- [64] Matthew Tancik, Pratul Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. *Advances in neural information processing systems*, 33:7537–7547, 2020.
- [65] GigaBrain Team, Boyuan Wang, Bohan Li, Chaojun Ni, Guan Huang, Guosheng Zhao, Hao Li, Jie Li, Jindi Lv, Jingyu Liu, et al. Gigabrain-0.5 m*: a vla that learns from world model-based reinforcement learning. *arXiv preprint arXiv:2602.12099*, 2026.
- [66] Robbyant Team, Zelin Gao, Qiuyu Wang, Yanhong Zeng, Jiapeng Zhu, Ka Leong Cheng, Yixuan Li, Hanlin Wang, Yinghao Xu, Shuailei Ma, et al. Advancing open-source world models. *arXiv preprint arXiv:2601.20540*, 2026.
- [67] Stephen Tian, Chelsea Finn, and Jiajun Wu. A control-centric benchmark for video prediction. *arXiv preprint arXiv:2304.13723*, 2023.
- [68] Naftali Tishby and Noga Zaslavsky. Deep learning and the information bottleneck principle. In *2015 IEEE information theory workshop (itw)*, pages 1–5. Ieee, 2015.
- [69] Samuel Triest, Matthew Sivaprakasam, Sean J Wang, Wenshan Wang, Aaron M Johnson, and Sebastian Scherer. Tartandrive: A large-scale dataset for learning off-road dynamics models. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 2546–2552. IEEE, 2022.
- [70] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- [71] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [72] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th International Conference on Machine Learning, ICML '08*, page 1096–1103. Association for Computing Machinery, 2008.
- [73] Youpeng Wen, Junfan Lin, Yi Zhu, Jianhua Han, Hang Xu, Shen Zhao, and Xiaodan Liang. Vidman: Exploiting implicit dynamics from video diffusion model for effective robot manipulation. *Advances in Neural Information Processing Systems*, 37:41051–41075, 2024.
- [74] Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149*, 2015.
- [75] World Labs. Marble: A multimodal world model. <https://www.worldlabs.ai/blog/marble-world-model>, 2025.
- [76] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*, pages 2226–2240. PMLR, 2023.
- [77] Zebin Xing, Xingyu Zhang, Yang Hu, Bo Jiang, Tong He, Qian Zhang, Xiaoxiao Long, and Wei Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1602–1611. IEEE, 2025.
- [78] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [79] Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation Alignment for Generation: Training Diffusion Transformers Is Easier Than You Think, October 2024.
- [80] Tianhe Yu, Ted Xiao, Austin Stone, Jonathan Tompson, Anthony Brohan, Su Wang, Jaspiar Singh, Clayton Tan, Dee M, Jodilyn Peralta, Brian Ichter, Karol Hausman, and Fei Xia. Scaling Robot Learning with Semantically Imagined Experience, February 2023.
- [81] Fatemeh Zargarbashi, Dhruv Agrawal, Jakob Buhmann, Martin Guay, Stelian Coros, and Robert W Sumner. Vq-style: Disentangling style and content in motion with residual quantized representations. *arXiv preprint arXiv:2602.02334*, 2026.
- [82] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018.
- [83] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

- [84] Ruijie Zheng, Jing Wang, Scott Reed, Johan Bjorck, Yu Fang, Fengyuan Hu, Joel Jang, Kaushil Kundalia, Zongyu Lin, Loïc Magne, et al. Flare: Robot learning with implicit world modeling. In *Conference on Robot Learning*, pages 3952–3971. PMLR, 2025.
- [85] Yinan Zheng, Ruiming Liang, Kexin Zheng, Jinliang Zheng, Liyuan Mao, Jianxiong Li, Weihao Gu, Rui Ai, Shengbo Li, Xianyuan Zhan, et al. Diffusion-based planning for autonomous driving with flexible guidance. In *International Conference on Learning Representations*, volume 2025, pages 37207–37227, 2025.
- [86] Allan Zhou, Moo Jin Kim, Lirui Wang, Pete Florence, and Chelsea Finn. NeRF in the Palm of Your Hand: Corrective Augmentation for Robotics via Novel-View Synthesis, January 2023.
- [87] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World Models on Pre-trained Visual Features enable Zero-shot Planning, November 2024.
- [88] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.
- [89] Haoran Zhu, Zhenyuan Dong, Kristi Topollai, and Anna Choromanska. AD-L-JEPA: Self-Supervised Spatial World Models with Joint Embedding Predictive Architecture for Autonomous Driving with LiDAR Data, January 2025.

Appendix

Table 7 summarizes how Hydra relates to prior navigation baselines across five dimensions – observation input, goal input, conditioning input, output modalities, and navigation policy type – extending the architectural comparison of Fig. 2.

Table 7: Comparison with visually-grounded robot navigation baselines. **I**: Image, **P**: Pose, **A**: Action.

Method	Obs. Input	Goal Input	Cond. Input	Output	Nav. Policy
GNM [57]	I	I	I	P	Reactive
VINT [58]	I	I	I	P	Reactive
NOMAD [61]	I	I	I	P	Reactive
VertiFormer [44]	I, A, P	I, P	A or P	I, A, P	Planning
NWM [6]	I	I	P	I	Planning
Hydra	I, A, P	I, P	A or P	I, A, P	Planning

A Extended Results

A.1 Single-Step Image Quality

Following NWM, we evaluate single-step visual predictive fidelity on a held-out subset of the RECON dataset. We benchmark Hydra against NWM/B and VertiFormer. As shown in Table 8, Hydra achieves competitive structural reconstruction (DreamSim, LPIPS) despite a smaller parameter footprint and encoding multiple modalities with only 50 NFEs compared to NWM’s 250 NFEs. While VertiFormer achieves the highest PSNR, this metric is notoriously misaligned with dynamic perception: deterministic models minimize MSE by averaging all possible futures, resulting in severe “ghosting” artifacts and blurry textures [40, 39]. PSNR strictly penalizes pixel-level noise but is blind to structural semantics. By leveraging Flow Matching over a discrete topological bottleneck, Hydra avoids regression-to-the-mean, preserving sharp, semantically valid structures and yielding superior perceptual scores (LPIPS, DreamSim).

Table 8: Offline single-step image quality on a held-out subset of the RECON dataset. Hydra achieves competitive spatial accuracy with fewer parameters and NFEs.

Model	Params	Dreamsim ↓	LPIPS ↓	PSNR ↑
NWM/B	200M	0.2480 ± 0.1265	0.5168 ± 0.1147	10.69
VertiFormer	27.11M	0.3738 ± 0.0560	0.5417 ± 0.0757	18.29
Hydra	143.29M	0.1103 ± 0.0345	0.2809 ± 0.0679	16.20

A.2 Long-Horizon Qualitative Comparison

Figure 5 shows the qualitative counterpart to the FVD comparison in Table 1 (Sec. 4.1). NWM initially renders high-fidelity frames, but its continuous latent space suffers from compounding errors; after approximately 8 seconds, the visual context rapidly degrades and decouples from the commanded actions (hallucination). Hydra’s discrete intent manifold adheres accurately to the physical commands over the full 16-second horizon.

A.3 Analysis of the Kinodynamic Codebook

While continuous action space preserves infinite-resolution control, which is highly beneficial for fine-grained maneuvers if the transition model is perfect, it is catastrophically sample-inefficient. Hydra mitigates the loss of fine-grained continuous resolution by using continuous Flow-Matching during final execution. These primitive discretized actions can be considered the mean of the trajectory decoded by the FM head. We analyze the 1-step spatial geometry of the learned discrete pose codebook, $\mathcal{V}_p$. As shown in Figure 6, the codebook captures a diverse, unnormalized distribution of spatial displacements ($\Delta X, \Delta Y$).

On the other hand, a predefined or uniformly spaced grid of actions is fixed independently of the world model. Because such a grid is never trained jointly with the model’s own representation, its entries have no defined relationship to the model’s embedding space at all – a hand-specified “turn left” primitive is not a point the encoder ever produced, so there is no meaningful notion of its distance to anything the model has learned. By contrast, because Hydra’s codebook is learned end-to-end alongside perception and dynamics, every entry is, by construction, a point the model itself produced



Figure 5: Comparison of 16-second video generation on a held-out subset of RECON. While NWM’s continuous latents degrade and drift from the commanded actions over time, Hydra’s discrete manifold maintains strict temporal grounding.

and can situate relative to its own manifold. The distance metric between codebook entries is therefore meaningful only because the codebook and the world model share one coupled representation. This is the same measure of topological safety that the KPC cost uses to evaluate candidate trajectories in the latent space without decoding during the planning phase.

The Flaw of Action Normalization. Generalist navigation models (e.g., GNM, ViNT, NoMaD, and NWM) universally normalize actions by the maximum speed of the specific robot platform to achieve dataset-agnostic training. However, this normalization destroys the absolute scale of the physical world; a maximum-throttle command for a slow indoor crawler becomes mathematically indistinguishable from a maximum-throttle command for a high-speed off-road ATV. Consequently, the generative model becomes blind to the physical constraints of the environment.

Emergent Physical Grounding. By using unnormalized actions, Hydra’s codebook naturally segregates physical domains. Low-speed indoor datasets (e.g., SACSON, RECON, SCAND) populate the dense inner clusters of $\mathcal{V}_p$ ($< 0.5m$), while high-speed off-road datasets (e.g., TartanDrive) form the sparse outer bounds ($> 1.0m$). Because the observation history $\mathcal{H}_t$ is contextually fused with these absolute velocities, Hydra learns a strict correlation between visual textures and physical speed limits.

This unnormalized grounding explains a profound emergent behavior in Hydra: *Contextual Manifold Shift*. If the model is conditioned on an indoor visual history (implying a slow-moving platform like a Clearpath Jackal) but is suddenly forced to execute a high-speed action intent (e.g., 1.5 m/step), the model will abruptly “hallucinate” a transition to an off-road TartanDrive environment. Rather than a standard generative failure, this is a resolution of conflicting modalities: the generative solver recognizes that the commanded velocity has zero probability density within the indoor contextual

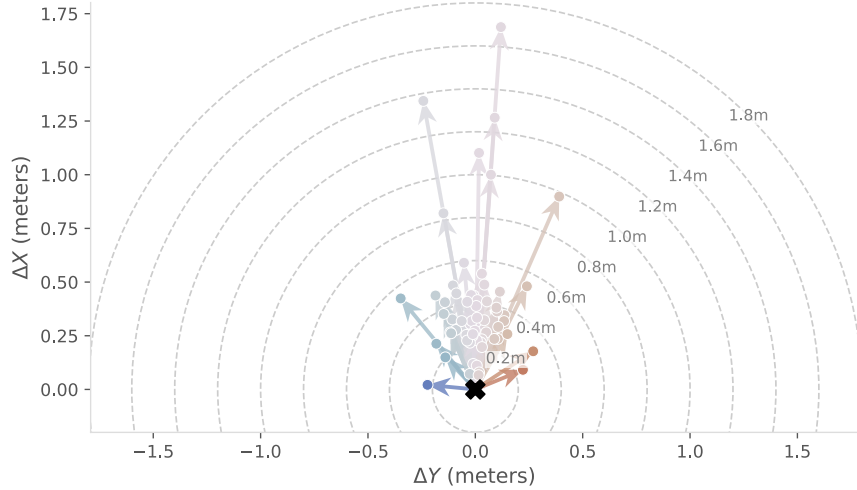


Figure 6: 1-step spatial coverage of the learned pose codebook $\mathcal{V}_p$. Unlike normalized baselines, the discrete manifold explicitly captures the absolute physical scaling of different robotic platforms, ranging from low-speed indoor maneuvers (dense central clusters) to high-speed off-road dynamics (sparse outer bounds).

manifold, and optimally shifts the predicted visual flow to the only environmental sub-manifold where that velocity is physically valid.

By bounding trajectory searches to this physically grounded codebook (Eq. 4), Hydra computationally prevents the execution of these OOD speed-context mismatches, ensuring safety in a way normalized models cannot. Furthermore, knowing which codes correspond to high-speed or turning, we can effectively change the robot’s behavior during deployment by masking specific codes. For example, masking high-speed codes prevents the robot from driving fast, or masking right turns prevents the robot from turning right.

A.4 Visualizing Latent Search: RViz Branch Trees

To make Discrete Latent Planning’s search process (Algorithm 1, Appendix E) inspectable, we visualize it live in RViz during physical deployment.



Figure 7: Live RViz visualization of Hydra’s Discrete Latent Planning search on the physical robot in a never-before-seen environment. **Left:** the robot’s current ego-centric observation. **Middle:** a subset of Hydra’s predicted future frames. **Right:** the corresponding candidate trajectories (green) rendered as a branching tree in the robot’s local frame; each branch is one topological maneuver considered before $\mathcal{J}_{KPC}$ selects the lowest-cost (purple) candidate for execution.

Figure 7 shows a representative search: the branch tree fans out from the robot’s current pose into a small number of distinct topological trajectories reflecting the discrete codebook’s role in constraining candidates to a small vocabulary of plausible maneuvers (Sec. A.3) rather than an unconstrained continuous spread.

A.5 Failure Analysis

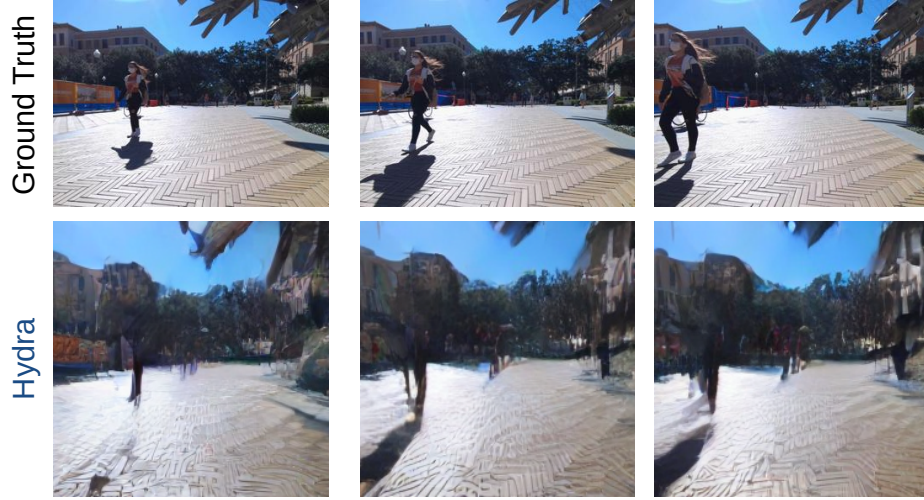


Figure 8: Vanishing humans in future predictions on SCAND. Top: ground-truth future frames, in which a pedestrian remains visible throughout. Bottom: [Hydra](#)’s predicted frames for the same sequence, in which the pedestrian is progressively smoothed into the background and disappears entirely by the final frame.

Failure mode: missed pedestrians under codebook quantization. Figure 8 shows a representative failure mode on SCAND’s social navigation scenarios: pedestrians present in the ground-truth future are absent from [Hydra](#)’s prediction. We attribute this to the limited cardinality of the visual codebook ($|\mathcal{V}_i| = 2048$), which acts as a stochastic low-pass filter: humans occupy a small subset of pixels relative to the static scene, and are discarded as high-frequency detail during quantization, the same limitation noted in our main-text discussion of limitations. This issue exists in NWM as well, but at a lower frequency. Incorporating human-centric object detection or increasing visual codebook capacity are natural directions to address this in future work.

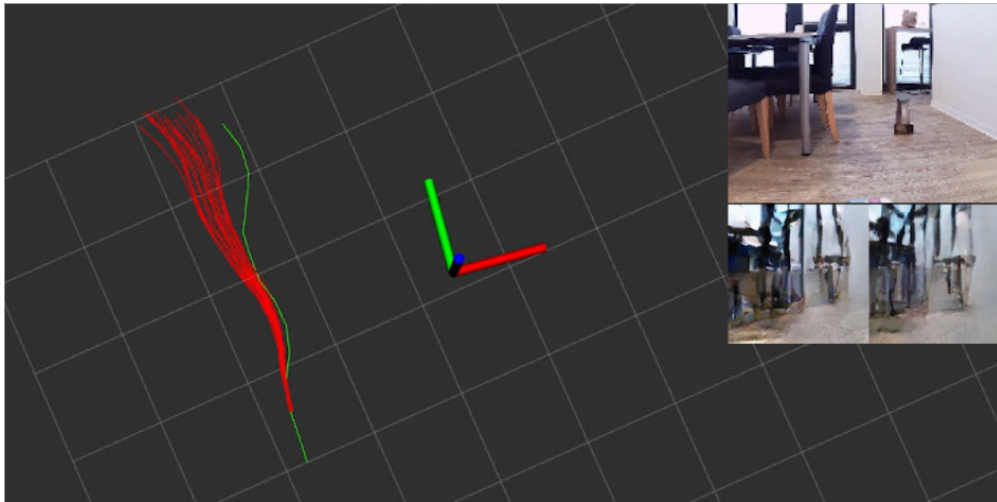


Figure 9: Goal-blind seed initialization causes a missed turn. The green trajectory is the path selected by the DLP cost among the sampled candidates at convergence; the seed distribution that generated the candidates was conditioned only on local scene context, not on the goal, delaying goal-aware correction until it is too late to complete the turn.

Failure mode: goal-blind seed initialization. In Fig. 9, the green trajectory shows the path selected by the DLP cost among the sampled candidates at convergence. This failure arises from seed conditioning: the seed distribution is generated without knowledge of the goal, so goal information only enters the optimization indirectly, through the

Euclidean-distance term of the KPC cost, which is evaluated *after* each iteration’s candidates are scored and only biases the seed population for the *next* iteration. With $N = 3$ iterations, set by the real-time planning budget, this leaves little runway for goal information to propagate before the robot must act. When a required turn falls close to the start of the planning horizon, the robot can physically pass the turning point before the distance signal has had enough iterations to steer the search toward it.

Design tradeoff: goal-conditioned vs. unbiased seeding. This failure mode reflects a more general tension between convergence speed and search diversity. We conditioned the seed on the goal direction. This resolved the near-turn failure directly, since the search no longer needed multiple iterations to discover the goal direction. However, it introduced a different problem: with all seeds already pointed at the goal, the sampled candidates clustered close together in trajectory space, reducing the diversity of the search and limiting exploration of the KPC cost landscape. We chose unbiased seed conditioning as the more conservative choice, accepting the near-turn failure mode described above in exchange for preserving exploration diversity across the sample population. We leave resolving this tradeoff, for instance via coarser goal-direction (rather than full goal-pose) conditioning, or an explicit diversity term applied alongside goal-conditioning, to future work.

B Learning Perspective on Hydra

As shown in Figure 1, Hydra moves planning from an external module that queries a separately-trained world model to a planner that lives inside the world model itself. We discuss the different parts of Hydra from a learning perspective that aided this transition.

Joint Distribution vs. Marginal Alignment. From a representation learning perspective [13], traditional late-fusion architectures optimize isolated marginal distributions and attempt to align them post-hoc [35] to prevent the dominant modality’s gradients from overcoming the other [38], often requiring complex contrastive losses. TUNA-2 [38] showed that this causes representation misalignment, however, given a large amount of data, the model can learn to align different modalities end-to-end. VertFormer [44] showed that, by employing early fusion, Hydra is able to approximate the joint multimodal distribution $p(I, p, a)$ on limited data. This imposes a strong inductive bias on the Spatio-Temporal Transformer: the self-attention mechanism must implicitly learn the underlying physical correlations (*e.g.*, forward kinematics and inverse dynamics) directly within the unified token sequence. Furthermore, the curriculum-based random infilling acts as a generalized Denoising Autoencoder objective [72], theoretically guaranteeing that the network learns robust, invariant representations that smoothly interpolate across missing sensor modalities.

Information Bottleneck and Posterior Collapse. The necessity of the discrete VQ bottleneck can be analyzed through the lens of the Information Bottleneck principle [68]. Unbounded continuous latent spaces in autoregressive or generative sequence models are notoriously prone to posterior collapse or high-frequency hallucination, as the model can perfectly memorize noisy inputs without extracting underlying causal structure. By forcing the continuous intent through a fixed-size VQ codebook, we strictly bound the mutual information between the observation history and the generated future. This information constraint forces the network to discard low-level motor noise and compress interactive dynamics into a compact manifold of semantically meaningful, reproducible kinodynamic primitives.

With this objective in mind, because these codebook embeddings are initialized randomly, passing them directly into an uncalibrated continuous differential equation solver causes rapid divergence. Therefore, we introduce a deterministic linear anchoring that ensures the codebook manifold remains stable and geometrically bounded by preventing unconstrained dimensional expansion [27] while the downstream generative ODE learns to map this intent to continuous trajectories. Before the flow-matching stage, we apply an MSE commitment loss to align the transformer output and the embedding of the ground truth modality. This is similar to pretraining the model on next-frame prediction, then fine-tuning it to predict high-quality images using flow matching; however, we do both stages at the same time, mimicking the REPA [79] regularization term.

A deep, non-linear projection would possess sufficient capacity to untangle a disorganized latent space [15], inadvertently permitting the VQ codebook to remain uninformative and unstructured, whereas a linear projection forces implicit subspace whitening on the discrete codebook [14, 19]. Furthermore, a linear projection mathematically compels the discrete embeddings to construct a highly structured, linearly separable manifold early in the training process. This constrained ‘tight grip’ ensures that the conditioning signal provided to the subsequent Flow Matching module acts as a robust, geometrically meaningful prior. We empirically validate the necessity of this strict linear information bottleneck in Section D.1, demonstrating that it significantly improves codebook utilization and training convergence.

Optimal Transport and ODE Stability. While recent generative policies heavily rely on score-based Diffusion Models [16], diffusion is formulated as a Stochastic Differential Equation (SDE) that suffers from highly curved sampling trajectories. Hydra utilizes Rectified Flow Matching [36], which frames the generative process as an

Ordinary Differential Equation (ODE). From an optimization standpoint, mapping a Gaussian prior directly to the target distribution via a linear velocity field $u(\psi_s) = \psi_1 - \psi_0$ implicitly encourages Optimal Transport (OT) [34]. This OT property results in straighter, deterministic probability paths during inference. For robotics, this is a critical theoretical advantage: straighter ODE paths allow the model to be integrated with significantly fewer Number of Function Evaluations (NFEs) without massive discretization error, directly enabling the real-time inference speeds required for physical deployment. This is in contrast with NWM, which formulates the generations as an SDE.

The Perceptual Fallacy: Limitations of LPIPS. Prior image-goal navigation frameworks utilizing world models often rely on visual similarity metrics to evaluate predicted trajectories. NWM, for instance, utilizes Learned Perceptual Image Patch Similarity (LPIPS) to calculate the distance between the predicted visual future and the goal image.

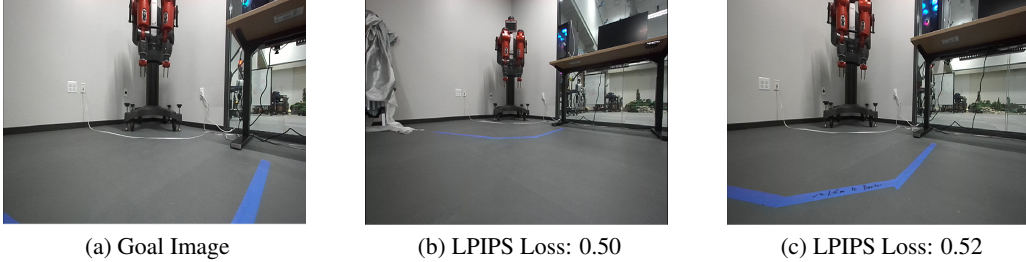


Figure 10: LPIPS deviation from geometric grounding. Note, these are not generated images.

C Architecture and Implementation Details

The architecture is modularly separated into the visual-proprioceptive encoders, the unified spatio-temporal TransformerEncoder, the discrete codebooks, and the multimodal latent flow decoders.

Visual Encoder. To compress high-resolution visual observations $I_t \in \mathbb{R}^{3 \times 224 \times 224}$ into spatial latents, we used the pretrained Stable Diffusion AutoencoderKL, which produces latents $z_i \in \mathbb{R}^{4 \times 28 \times 28}$, and kept it frozen during training.

Infilling Curriculum Strategy. Rather than employing a naive hard-switching dropout strategy introduced in [44], we implement a cosine-annealed modality infilling scheduler. During the initial epochs, the network is trained on the fully observable joint distribution. As training progresses, the masking probability for pose and action tokens is smoothly

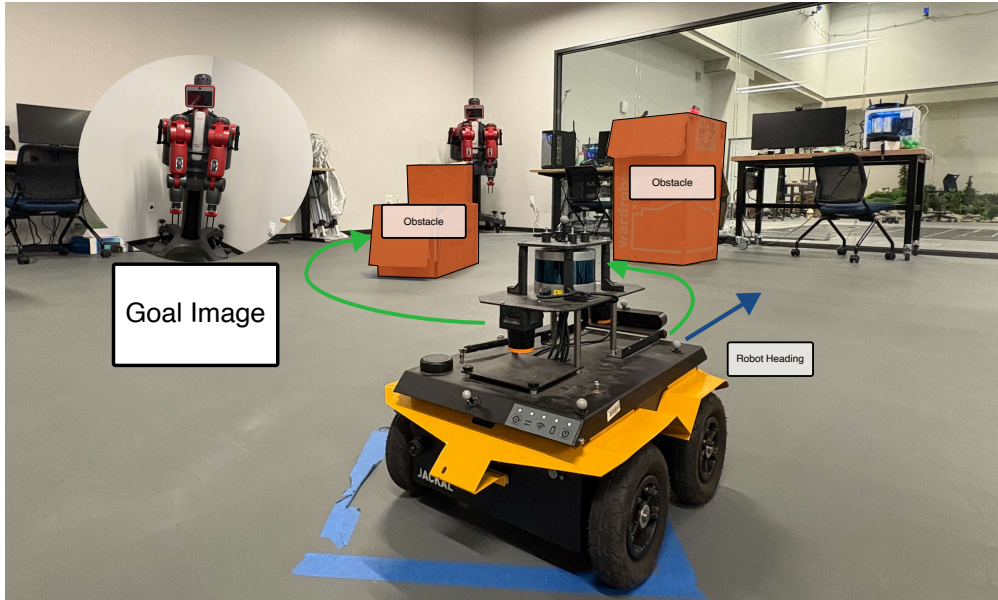


Figure 11: Physical experiment setup with obstacles.

increased following a cosine curve, preventing gradient shock and teaching the early-fusion Transformer to robustly infer missing modalities without destabilizing the latent flow.

Over the first $E_{warmup} = 10$ epochs, we gradually decay the probability of the network receiving all modalities (the fully observable joint distribution, p_{joint}) from 0.5 to 0.1. For a given epoch e , the probabilities are defined as:

$$p_{joint} = 0.1 + 0.4 \left[\frac{1}{2} \left(1 + \cos \left(\pi \min \left(1, \frac{e-1}{E_{warmup}} \right) \right) \right) \right] \quad (7)$$

The remaining probability mass is split evenly between the Forward Kinodynamics (FKD) mask state and the Inverse Kinodynamics (IKD) mask state:

$$p_{fkd} = p_{ikd} = \frac{1 - p_{joint}}{2} \quad (8)$$

This soft-switching curriculum ensures that the early-fusion Transformer learns the fully observable transition dynamics before being forced to infer missing marginals.

Early Fusion. To early fuse modalities before passing them to the Transformer, Hydra simply sums all the modalities to create a unified representation without introducing extra parameters, and then adds sinusoidal positional encoding [71]. This is due to cutting unnecessary parameters and easier reconstruction of the modality in the output by extracting the modality-specific information from the stream.

Flow Matching Head. To synthesize continuous trajectories and high-fidelity visual latents from the discrete intent vectors, Hydra employs a conditional Rectified Flow Matching [36] framework modulated via AdaLN-Zero. We deliberately tailor the representational capacity of the Flow Matching network to the specific spatial-temporal and latency constraints of each modality.

For the high-dimensional visual world model, we utilize a full Diffusion Transformer (DiT) block [51], allowing spatial patches to continuously query each other via Multi-Head Self-Attention (MSA) to maintain structural coherence across time. Conversely, the action and pose Flow Matching heads strictly demand high-frequency inference, where the $\mathcal{O}(N^2)$ computational bottleneck of iterative self-attention is prohibitive. To achieve real-time control, the action and pose heads are stripped of MSA, utilizing strictly Pointwise AdaLN-Zero MLPs.

Datasets. To train Hydra’s multimodal representations, we utilize the heterogeneous robotic dataset corpus [57] containing 100,355 samples, broken down into 157,356,640 tokens seen during training. This corpus encompasses a diverse set of environments, including off-road [69], indoor [22], outdoor [56], and social [28] navigation scenarios. The dataset is structured as synchronized temporal tuples of $\{I_t, p_t, a_t\}$, providing the exact structural alignment required to jointly optimize Hydra’s early-fusion latent manifold across visual, spatial, and kinematic domains. Unlike NWM and GNM, we *do not* standardize the step size across agents by dividing the distance agents travel between frames by their maximum step size in meters, so that the model cannot discriminate between lower-speed robots like Clearpath Jackal and high-speed platforms such as the ATV used in TartanDrive; instead, we rely on the codebooks to capture these differences, as shown in Sec. A.3. We filter out backward movements, following GNM [57].

Training Hyperparameters. The model is trained end-to-end (with the AutoencoderKL frozen) using the AdamW optimizer. Table 9 reports the complete hyperparameter set.

Variable Target Horizon. While the maximum prediction horizon is fixed to 3 seconds ($H = 12$ steps at 4Hz), we utilize a variable-length temporal regularization strategy during training. For each batch, the target rollout length is uniformly sampled between 1 and 3 seconds. This prevents the model from overfitting to a rigid temporal boundary, ensuring robust transition dynamics at arbitrary planning horizons during physical deployment.

D Architectural Ablations

To isolate the specific contributions of Hydra’s architectural components, we perform a series of ablation studies focusing on the discrete intent bottleneck, context compression, and inference memory constraints.

D.1 Projection Capacity and Codebook Utilization

To empirically validate the necessity of the strict linear information bottleneck preceding the Flow Matching solver, we ablated the projection layer architecture. We compared our standard linear projection against a higher-capacity 2-layer MLP with GeLU activation. We evaluated the models at 237K training steps, analyzing both codebook perplexity (the entropy of discrete code utilization) and the cosine similarity of the multi-task gradients.

While both architectures achieved 100% code usage, the linear projection yielded a significantly higher image codebook perplexity (608.36) compared to the MLP (465.42). This indicates that the non-linear MLP possesses enough capacity

Table 9: Hyperparameters and architectural details of Hydra. The model utilizes a Transformer backbone combined with dimensionally constrained, modality-specific VQ codebooks and Flow Matching decoders.

Parameter	Value
Training & Optimization	
Optimizer	AdamW
Learning Rate	2×10^{-4}
Weight Decay	0.08
Batch Size	8
Gradient Accumulation Steps	32
Total Effective Batch Size	256
Epochs	60
Gradient Clipping	1.0
Variable Prediction Horizon	4 . . . 12 steps
Dataset Parameters	
Observation Length (H_{obs})	8 steps
Prediction Length (H_{pred})	12 steps
Image Size	224×224
Patch Size	2×2
Unified Transformer Architecture	
Visual Backbone	SD-VAE-FT-EMA
Positional Encoding	Sinusoidal 3D
Transformer Layers (Encoder / Decoder)	6 / 6
Transformer Dimension (d_{model})	768
Attention Heads	12
Feedforward Dimension	2048
Dropout	0.1
Vector Quantization (VQ) Bottleneck	
Image Codebook Size ($\mathcal{C}_{img}$)	2048
Action Codebook Size ($\mathcal{C}_{act}$)	64
Pose Codebook Size ($\mathcal{C}_{pose}$)	64
Codebook Dimension	768
Commitment Cost (β)	0.25
EMA Decay (γ)	0.99
Flow Matching Heads	
ODE Block Architecture	DiT block
Hidden Dimension	512
Sampling Steps, Image Head (NFEs)	50
Sampling Steps, Pose/Action Heads	10
ODE Source Distribution	$\mathcal{N}(0, \mathbf{I})$
Deterministic Loss Weight	0.1

to lazily route information, resulting in an unstructured, low-entropy utilization of the visual manifold. Conversely, the strict linear projection forces the network to uniformly distribute representations across the available codebook.

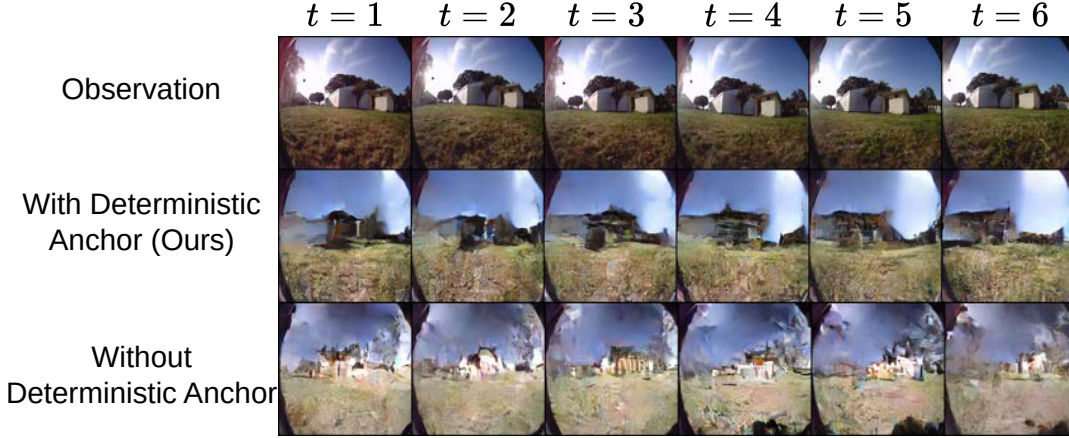


Figure 12: Early Training Dynamics (2.1K Steps). A comparison between the model’s early-stage generation, both with (middle) and without (bottom) the deterministic anchor head. While the bottom row suffers from spatial smearing, the model utilizing the deterministic head shows a better ability to construct structural details, like the building’s edges against the sky, even after very limited training.

D.2 Deterministic Decoder Heads

Besides the projection capacity, to empirically validate the necessity of this deterministic projection preceding the VQ codebooks, we visualized the early training dynamics (at 2.1K steps) with and without this auxiliary head. As shown in Figure 12, passing randomly initialized codebooks directly into the continuous flow-matching solver causes severe representational instability. Conversely, incorporating the deterministic head provides an immediate stabilizing prior. By grounding the codebooks with a standard MSE commitment loss, the network is forced to maintain spatial coherence early in the training process, preventing the downstream generative ODE from diverging and accelerating the formation of structural scene details.

E Latent Planning Algorithm Details

E.1 Two Sources of Stochasticity

Generating diverse candidate trajectories from a discrete VQ latent space requires randomizing two different things, at two different stages of the search.

Why codebook-level noise alone is not enough. A natural first attempt is to compute the Transformer’s intent tokens once, then let each of the B candidates independently sample a codebook entry from the resulting logits. This produces candidates that differ only in *which nearby codebook vector each one snaps to* – and because the codebook is trained so that nearby vectors are semantically similar, swapping the 1st-nearest for the 3rd-nearest vector rarely changes the decoded trajectory’s shape in any meaningful way. This gives quantization jitter around a single underlying idea (e.g., “swerve left” vs. “swerve left, very slightly differently”), never qualitatively different intents (e.g., “swerve left” vs. “swerve right”).

Site 1: Gaussian noise on the continuous intent query. To obtain genuinely different trajectory shapes, the input to the Transformer decoder itself must change, since the Transformer – not the codebook – decides the high-level shape of the candidate. We therefore perturb the continuous intent query independently for each candidate before it is passed to the decoder:

$$\tilde{\epsilon}_b[t] = \alpha_{sm} \cdot \epsilon_b[t] + (1 - \alpha_{sm}) \cdot \tilde{\epsilon}_b[t - 1], \quad \epsilon_b \sim \mathcal{N}(0, I), \quad (9)$$

smoothed across the horizon via an Exponential Moving Average (EMA) with coefficient α_{sm} so that a perturbation at one timestep biases neighboring timesteps in a similar direction – producing a smooth alternate path (e.g., “curve further right the whole way”) rather than step-to-step jitter that would be physically implausible for a robot to execute. Each candidate’s noisy query, $q_b^{(i)} = P_{intent}^{(i-1)} + \sigma^{(i)} \tilde{\epsilon}_b$, is then passed through a fresh Transformer forward pass, yielding B genuinely different sets of intent tokens rather than B discretizations of one shared set. The noise scale $\sigma^{(i)}$ shrinks geometrically across iterations, giving the search a coarse-to-fine, CEM-like character: broad exploration early, local refinement late.

Site 2: Top- K Gumbel-max discretization. Each candidate’s (now distinct) continuous intent tokens must still be snapped onto valid codebook vectors before decoding. We do this stochastically, restricted to the K nearest vectors by cosine similarity, rather than with a hard nearest-neighbor snap – adding a second, finer layer of exploration around each of Site 1’s hypotheses while the top- K restriction keeps candidates from drifting to codebook entries that are cosine-far, *i.e.*, off-manifold and physically implausible (Sec. A.3). Like Site 1, the raw Gumbel noise driving this discretization is itself temporally smoothed via its own EMA, with its own coefficient α_{gum} , independent of Site 1’s α_{sm} : without this, independent per-timestep Gumbel draws would let each candidate’s discretization jitter step to step even when its underlying continuous trajectory (from Site 1) is smooth. This is detailed in the remainder of this section.

Both sites are necessary and serve different purposes: Site 1 diversifies *what the model considers*; Site 2 makes each consideration’s discretization robust rather than collapsed onto a single nearest neighbor, while remaining temporally coherent in its own right. Algorithm 1 uses both simultaneously, each independently smoothed and annealed.

Algorithm 1 Discrete Latent Planning (Hydra)

Require: Observation history $\mathcal{H}_t$, goal pose P_{goal} , visual goal z_{goal} , horizon T , candidates B , iterations N

- 1: $h_{ctx} \leftarrow \text{UnifiedEncoder}(\mathcal{H}_t)$ ▷ encode history once
- 2: $J_{best} \leftarrow \infty$, $c_{best} \leftarrow$ unbiased expert prior
- 3: **for** $i = 0$ to $N - 1$ **do**
- 4: Generate B candidate queries around c_{best} , adding decaying Gaussian noise (unperturbed on the first pass) ▷ explore trajectory space
- 5: **for** $b = 1$ to B **do**
- 6: $z_{e,b} \leftarrow \text{TransformerDecoder}(h_{ctx}, q_b)$
- 7: $c_b \leftarrow$ discretize $z_{e,b}$ into a codebook index via Top- K -restricted, annealed Gumbel-Softmax sampling ▷ differentiable discretization
- 8: $\hat{P}_b, \hat{z}_{img,b} \leftarrow c_b$ to pose and visual codes
- 9: $J_{total,b} \leftarrow \text{KPC}(\hat{P}_b, \hat{z}_{img,b}, P_{goal}, z_{goal})$ ▷ Kinematic-Perceptual Cost, Eq. (4)
- 10: **end for**
- 11: $b^* \leftarrow \arg \min_b J_{total,b}$
- 12: **if** $J_{total,b^*} < J_{best}$ **then**
- 13: $J_{best} \leftarrow J_{total,b^*}$; $c_{best} \leftarrow c_{b^*}$
- 14: **end if**
- 15: $current_intent \leftarrow$ momentum update toward $\hat{P}_{b^*}$ ▷ shift search region toward elite
- 16: **end for**
- 17: **return** FlowDecoder(Lookup(c_{best} , $\mathcal{V}_p$)) ▷ high-fidelity continuous execution

Note that J_{best} and c_{best} track the globally best candidate across *all* N iterations.

Standard Gumbel-Max Trick. Let $s_{t,c}$ denote the unnormalized logit predicted by the Transformer decoder for timestep $t \in \{1, \dots, T\}$ and codebook entry $c \in \mathcal{V}_s$. Sampling from the Categorical distribution $p(c_t) \propto \exp(s_{t,c}/\rho)$ with temperature ρ is equivalent to the Gumbel-Max trick:

$$c_t = \arg \max_{c \in \mathcal{V}} \left(\frac{s_{t,c}}{\rho} + g_{t,c} \right)$$

where $g_{t,c} \sim \text{Gumbel}(0, 1)$ is i.i.d. noise sampled via $g_{t,c} = -\log(-\log(u_{t,c}))$ with $u_{t,c} \sim \mathcal{U}(0, 1)$.

Iteration-Dependent Annealing Schedule. Since the search is an iterative optimizer running for N depth iterations, we anneal the Site 2 temperature ρ and Site 2’s own smoothing coefficient α_{gum} at each iteration i . Site 1’s smoothing coefficient α_{sm} is, by contrast, a fixed constant and is not annealed; only its noise *scale* $\sigma^{(i)}$ shrinks across iterations.

Simulated Annealing (Coarse-to-Fine Search): We exponentially decay the Site 2 temperature to transition from broad topological exploration to greedy exploitation:

$$\rho^{(i)} = \rho_0 \cdot 0.1^i$$

where ρ_0 is the base temperature.

Annealing the Gumbel Smoothing Factor: We linearly increase α_{gum} as iterations progress:

$$\alpha_{gum}^{(i)} = \min(0.4 + 0.15 \cdot i, 1.0)$$

During early iterations, $\alpha_{gum}^{(i)}$ is small, generating highly correlated Gumbel noise for large-scale, smooth topological exploration; in late iterations, $\alpha_{gum}^{(i)} \rightarrow 1.0$, allowing fine-grained, near-independent micro-adjustments to individual timesteps along the elite trajectory.

Top- K Kinematic Masking. Even with temporally correlated exploration, the network might occasionally sample outlier codes that violate kinematic constraints. We strictly bound the sampling space by projecting a hard constraint mask over the logits. Let $\mathcal{T}_t$ be the set containing the indices of the K largest logits at timestep t . We define the mask $\mathbf{M} \in \{0, -\infty\}^{T \times V}$ as:

$$M_{t,c} = \begin{cases} 0 & \text{if } c \in \mathcal{T}_t \\ -\infty & \text{otherwise} \end{cases}$$

This formulation guarantees that the sampled latent intent remains within the physically plausible manifold learned by Hydra (enforced by $\mathbf{M}$), while the auto-correlated Site 1 perturbation ensures that the sampled intents exhibit the temporal coherence necessary for stable robotic control.

E.2 Classifier-Free Guidance for Navigation Trajectory Generation

Vocabulary-based Maneuver Candidate Generation. To circumvent the OOD hallucination risk of latent extrapolation, we restrict the search space to the valid pose codebook $\mathcal{V}_p$. We observe that the codebook itself encodes a diverse maneuver vocabulary (Fig. 6): its entries, decoded as intents, span endpoint headings covering both turn signs. Given the mean-pooled unconditional prior $\bar{z}_0 = \frac{1}{K} \sum_{k=1}^K z_0^{(k)}$, we construct a batch of N candidate intents in a single forward pass as

$$\mathcal{C} = \underbrace{\{\bar{z}_0\}}_{\text{expert default}} \cup \underbrace{\{\mathbf{e}_c\}_{c \in \mathcal{V}_p}}_{\text{pure maneuvers}} \cup \underbrace{\{\bar{z}_0 + \lambda_j(\mathbf{e}_{c_j} - \bar{z}_0)\}}_{\text{convex mixes}}, \quad \lambda_j \sim \mathcal{U}[0.25, 1], \quad (10)$$

where $\mathbf{e}_c$ denotes codebook embedding c , broadcast as a constant intent over the horizon, and the candidate budget is split evenly between pure codebook entries and convex mixes that interpolate between the scene-conditioned prior and the vocabulary. Crucially, every candidate is re-quantized through the VQ bottleneck $\mathcal{Q}(\cdot)$ before decoding, so even the mixes are snapped back onto the discrete manifold; by construction only valid codes are ever decoded. Where stochastic diversity is desired, we add small Gaussian perturbations drawn *once per candidate and timestep*, shared across the K intent slots and smoothed over time by an exponential moving average. Sharing the noise across slots is essential: independent per-slot perturbations cancel under the mean-pooling aggregation by the central limit theorem, collapsing all candidates onto the expert default regardless of the noise temperature.

Deterministic Intent Decoding. Candidates are decoded with the deterministic pose head D_{pose} – the linear anchoring head trained on quantized intents (App. D.2) – rather than the flow-matching sampler. The deterministic head is pointwise stable, which makes candidate scores comparable across the batch, and it is trained exactly on the quantized intents it receives here. The stochastic ODE decode, in contrast, exhibits high per-seed variance for maneuver-code candidates that lie far from the prior mode, which would render selection meaningless. The same decode path is then used to execute the winner, so selection and execution are identical by construction. Since the deterministic decode is a single matrix multiplication per candidate, scoring N hypotheses is cheaper than a single flow-matching integration, preserving the ultra-low latency required for real-time control.

E.3 Sampling: Candidate Path Guidance

When the global planner supplies a reference path $\mathcal{R} = \{r_1, \dots, r_S\}$, we therefore score each candidate with a truncated Chamfer distance to $\mathcal{R}$. Let $\ell_t = \sum_{i \leq t} \|p_i - p_{i-1}\|_2$ be the candidate’s arc length and $L_{\mathcal{R}}$ the total arc length of the reference. We retain only the prefix $\mathcal{K} = \{t : \ell_t \leq L_{\mathcal{R}}\}$ and compute:

$$d_{\text{path}}(\tau) = \frac{1}{|\mathcal{K}|} \sum_{t \in \mathcal{K}} \min_s \|\tau_t - r_s\|_2 - w_{\text{prog}} \cdot \rho_{\text{reward}}(\tau), \quad \rho_{\text{reward}}(\tau) = \frac{\arg \min_s \|\tau_T - r_s\|_2}{S - 1}, \quad (11)$$

where each candidate point is matched to its *nearest* reference point, so candidates are not penalized for executing the path at a different speed than the reference, and $\rho_{\text{reward}}(\tau)$ is a progress reward measuring how far along $\mathcal{R}$ the candidate reaches. The path cost is blended with the candidate’s deviation from the unconditional expert trajectory $p_{1:T}^{\text{prior}}$:

$$\mathcal{J}_{\text{track}}(\tau) = \omega \cdot d_{\text{path}}(\tau) + \max(1 - \omega, 0) \cdot \underbrace{\frac{1}{T} \sum_t \|p_t - p_t^{\text{prior}}\|_2}_{\text{prior deviation}}, \quad (12)$$

where $\omega=1$ yields strict path following, $\omega<1$ increasingly trusts the obstacle-avoiding prior, and $\omega>1$ is permitted – the clamped second term vanishes, amplifying the path signal without rewarding absurd off-manifold decodes for their distance from the prior. The selected intent is decoded to continuous actions and executed directly.